Optimization 2

ISE 5406

Beyond Steepest Descent: CG, BFGS, and L-BFGS

Dr. Robert Hildebrand

Grado Department of Industrial & Systems Engineering
Virginia Tech

Roadmap

Why Do We Need These Methods?

Many of the most important problems in science and engineering reduce to solving

$$\min_{\mathbf{x}\in\mathbb{R}^n} f(\mathbf{x})$$

where n is enormous—often 10^5 to 10^9 variables. At this scale, Newton's method ($O(n^3)$ per iteration) is completely impractical, and even storing an $n \times n$ Hessian is impossible.

Key observation: Many of these problems are either convex quadratics $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A\mathbf{x} - \mathbf{b}^T\mathbf{x}$ or can be well-approximated locally by quadratics. Solving the quadratic case efficiently is therefore fundamental.

Minimizing $\frac{1}{2}\mathbf{x}^T A\mathbf{x} - \mathbf{b}^T\mathbf{x}$ is **equivalent** to solving the linear system $A\mathbf{x} = \mathbf{b}$. This equivalence connects optimization to linear algebra, and explains why the Conjugate Gradient method is simultaneously:

- An optimization algorithm (minimizing a quadratic)
- A linear solver (solving $A\mathbf{x} = \mathbf{b}$ without factoring A)

Where Do Large-Scale Problems Come From?

1. Discretizing partial differential equations (PDEs):

Finite element / finite difference methods for structural analysis, fluid dynamics, heat transfer, and electromagnetics produce sparse SPD systems $A\mathbf{x} = \mathbf{b}$ with $n = 10^5$–10^8. Direct solvers (Gaussian elimination) are too expensive—CG with a good preconditioner is the standard approach.

2. Machine learning and statistics:

Training logistic regression, conditional random fields, or fine-tuning neural networks requires minimizing a loss function over $n = 10^4$–10^9 parameters. L-BFGS is the optimizer of choice when full-batch gradients are available (e.g., `scipy.optimize`, `scikit-learn`).

3. Inverse problems and imaging:

Image reconstruction (MRI, CT, seismic tomography) involves solving $\min \|A\mathbf{x} - \mathbf{b}\|^2 + \lambda R(\mathbf{x})$ where A encodes the physics of the measurement. CG and L-BFGS dominate these applications.

4. Optimal control and engineering design:

Trajectory optimization, shape optimization, and topology optimization in aerospace, automotive, and biomedical engineering involve large-scale nonlinear programs where BFGS/L-BFGS are core solvers.

Why Not Just Factor A?

If $A\mathbf{x} = \mathbf{b}$ is a linear system and $A \succ 0$, why not use Cholesky factorization $A = LL^T$ and solve via back-substitution?

For small systems, you should! If $n \leq$ a few thousand:

- Cholesky costs $\frac{1}{3}n^3$ flops and gives the exact answer (up to machine precision)
- `numpy.linalg.solve` or `scipy.linalg.cho_solve` are highly optimized (LAPACK)
- No iteration, no convergence criteria, no line search—just a direct answer

The problem at scale. For $n = 10^6$ (a modest PDE mesh):

- A is $10^6 \times 10^6$ but sparse—perhaps 10 nonzeros per row ($\sim 10^7$ entries total)
- Cholesky on sparse A produces fill-in: L has far more nonzeros than A. For a 3D problem on an $n^{1/3} \times n^{1/3} \times n^{1/3}$ grid, the fill-in is $O(n^{4/3})$, requiring $O(n^{4/3})$ storage and $O(n^2)$ flops
- At $n = 10^6$: $\sim 10^8$ entries in L ($\sim$1 GB) and $\sim 10^{12}$ flops ($\sim$hours)
- At $n = 10^8$: completely infeasible for direct methods

CG exploits sparsity perfectly. Each CG iteration needs only one matrix-vector product $A\mathbf{d}_k$. If A has nnz nonzeros, this costs $O(\mathrm{nnz})$. Total storage is $O(n + \mathrm{nnz})$—CG never modifies or fills in A.

The Anytime Property: Stop Whenever You Want

A fundamental advantage of iterative methods over direct solvers:

Anytime Approximation
CG (and BFGS, L-BFGS) produce a sequence of improving approximations $\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \ldots$ At any iteration k, $\mathbf{x}_k$ is a valid approximate solution that you can use immediately.

Cholesky / LU: You get nothing useful until the factorization is complete. If you stop halfway through LL^T, you have a partially factored matrix that doesn't help solve the system. It's all-or-nothing.

CG: At iteration k, $\mathbf{x}_k$ minimizes $\phi(\mathbf{x})$ over the k-dimensional Krylov subspace. This means:

- After $k \ll n$ iterations, you may already have an excellent approximation
- The error bound $\|\mathbf{x}_k - \mathbf{x}^*\|_A \leq 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k \|\mathbf{x}_0 - \mathbf{x}^*\|_A$ tells you how fast
- With preconditioning ($\kappa(M^{-1}A) \ll \kappa(A)$), convergence to engineering accuracy ($\sim 10^{-6}$) often takes $\sim$50–100 iterations regardless of n

Why this matters in practice:

- Real-time systems: Use as many CG iterations as the time budget allows
- Newton-CG: Solve the quadratic subproblem approximately—the outer Newton iteration doesn't need an exact inner solve ("truncated Newton")
- Stochastic problems: When the right-hand side is noisy, solving to full precision wastes effort—a rough CG solution suffices

For $n > 10^4$ with sparse A: CG + preconditioner beats Cholesky in both time and memory. For $n > 10^6$: direct methods are simply not an option.

The Central Role of the Quadratic

Why does the quadratic $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A\mathbf{x} - \mathbf{b}^T\mathbf{x}$ keep appearing?

Directly: Many real problems *are* quadratic:

- Least squares: $\min \|A\mathbf{x} - \mathbf{b}\|^2 = \min \mathbf{x}^T A^T A\mathbf{x} - 2\mathbf{b}^T A\mathbf{x} + \mathbf{b}^T\mathbf{b}$
- Energy minimization in finite elements: stiffness matrix K, $\min \frac{1}{2}\mathbf{x}^T K\mathbf{x} - \mathbf{f}^T\mathbf{x}$
- Gaussian process regression: solving $(K + \sigma^2 I)\mathbf{x} = \mathbf{y}$

Indirectly: Newton's method at each iteration solves a quadratic model:

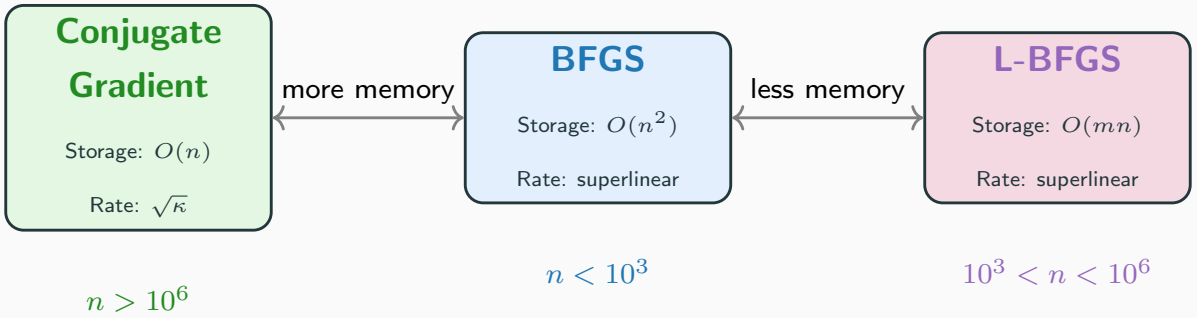
$$\min_{\mathbf{p}} \; f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^T\mathbf{p} + \tfrac{1}{2}\mathbf{p}^T\nabla^2 f(\mathbf{x}_k)\mathbf{p}$$

When n is large, we solve this approximately using CG—this is the Newton-CG method (truncated Newton).

Quasi-Newton methods (BFGS, L-BFGS) take this further: instead of solving the quadratic subproblem, they build the curvature model incrementally from gradient differences, avoiding the Hessian entirely.

Three Methods, One Goal

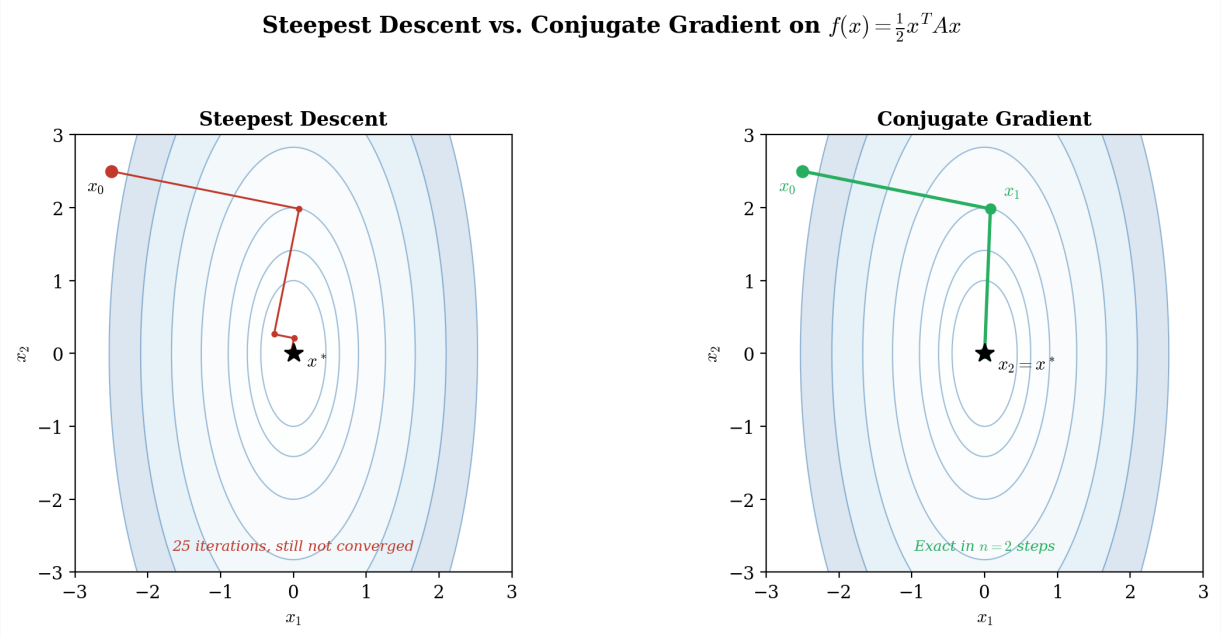
Goal: Solve $\min_{\mathbf{x}\in\mathbb{R}^n} f(\mathbf{x})$ using only gradient information.



Common thread: All three avoid computing the Hessian. They differ in how much curvature information they store and exploit:

- **CG:** Builds curvature implicitly via A-conjugate directions ($O(n)$ memory)
- **BFGS:** Maintains a full $n \times n$ inverse Hessian approximation ($O(n^2)$)
- **L-BFGS:** Stores m recent gradient pairs to approximate BFGS ($O(mn)$)

Why Steepest Descent Is Not Enough



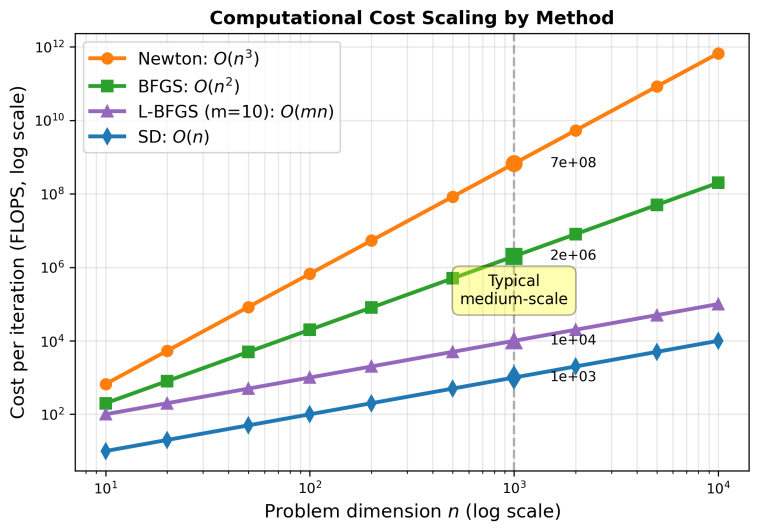
The problem: Steepest descent zigzags on ill-conditioned problems. Each step is orthogonal to the previous (exact line search forces $\mathbf{g}_{k+1}^T \mathbf{g}_k = 0$), but the iterates revisit the same subspaces.

The fix: All three methods—CG, BFGS, L-BFGS—incorporate curvature to avoid this pathology.

Python: Steepest Descent vs. Conjugate Gradient

```python
import numpy as np

# Ill-conditioned quadratic: f(x) = 0.5 x^T A x - b^T x
n = 100
eigvals = np.logspace(0, 4, n)        # kappa = 10^4
Q, _ = np.linalg.qr(np.random.randn(n, n))
A = Q @ np.diag(eigvals) @ Q.T
b = np.random.randn(n);  x0 = np.zeros(n)

# Steepest Descent with exact line search
def steepest_descent(A, b, x0, tol=1e-8, maxiter=5000):
    x = x0.copy()
    for k in range(maxiter):
        g = A @ x - b
        if np.linalg.norm(g) < tol: break
        alpha = (g @ g) / (g @ A @ g)
        x = x - alpha * g
    return x, k

# Linear Conjugate Gradient for Ax = b
def conjugate_gradient(A, b, x0, tol=1e-8):
    x = x0.copy()
    r = b - A @ x;  d = r.copy()
    for k in range(len(b)):
        if np.linalg.norm(r) < tol: break
        Ad = A @ d
        alpha = (r @ r) / (d @ Ad)
        x = x + alpha * d
        r_new = r - alpha * Ad
        beta = (r_new @ r_new) / (r @ r)
        d = r_new + beta * d;  r = r_new
    return x, k

_, sd_iters = steepest_descent(A, b, x0)
_, cg_iters = conjugate_gradient(A, b, x0)
print(f"SD: {sd_iters} iters,  CG: {cg_iters} iters")
# Typical output: SD: 5000 iters,  CG: 100 iters  (n steps!)
```

Computational Cost: The Full Picture



Method	Cost/iter	Memory	Convergence	Best for
Steepest Descent	$O(n)$	$O(n)$	Linear	prototyping
Conjugate Gradient	$O(n)$	$O(n)$	$\sqrt{\kappa}$-linear	$n > 10^6$
L-BFGS ($m \approx 10$)	$O(mn)$	$O(mn)$	Superlinear	$n \approx 10^3$–10^6
BFGS	$O(n^2)$	$O(n^2)$	Superlinear	$n < 10^3$
Newton	$O(n^3)$	$O(n^2)$	Quadratic	Hessian cheap

Roadmap

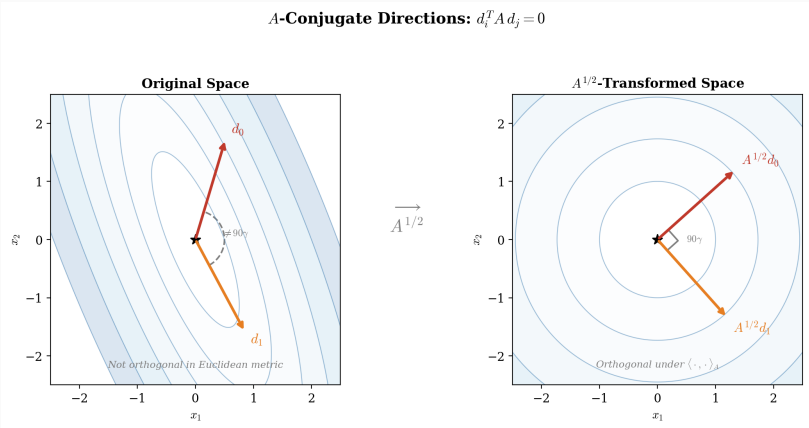
What Does "Conjugate" Mean?

Consider the quadratic $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A\mathbf{x} - \mathbf{b}^T\mathbf{x}$ with $A \succ 0$.

Definition (A-conjugacy)
Nonzero vectors $\mathbf{d}_0, \mathbf{d}_1, \ldots, \mathbf{d}_m$ are **A-conjugate** if

$$\mathbf{d}_i^T A\, \mathbf{d}_j = 0 \quad \text{for all } i \neq j$$

Geometric picture: A-conjugacy = orthogonality in the A-inner product $\langle \mathbf{u}, \mathbf{v} \rangle_A = \mathbf{u}^T A\mathbf{v}$.



After the change of variables $\mathbf{y} = A^{1/2}\mathbf{x}$, the elliptical contours become circles and A-conjugate directions become truly orthogonal.

Key Theorem: Finite Convergence

Theorem (Conjugate Direction Method)
Given n A-conjugate directions $\mathbf{d}_0, \ldots, \mathbf{d}_{n-1}$ and any starting point $\mathbf{x}_0$, the iteration

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k, \qquad \alpha_k = -\frac{\mathbf{g}_k^T \mathbf{d}_k}{\mathbf{d}_k^T A \mathbf{d}_k}$$

converges to $\mathbf{x}^* = A^{-1}\mathbf{b}$ in **at most n steps**.

Proof idea: Since $\{\mathbf{d}_0, \ldots, \mathbf{d}_{n-1}\}$ are A-conjugate, they form a basis for $\mathbb{R}^n$. Expand $\mathbf{x}^* - \mathbf{x}_0 = \sum_i \sigma_i \mathbf{d}_i$ and use A-conjugacy to show $\alpha_k = \sigma_k$ at each step.

The question: How do we construct A-conjugate directions using only matrix-vector products? The answer is the CG algorithm.

The CG Algorithm

Algorithm: Linear Conjugate Gradient (Hestenes–Stiefel, 1952)
Input: $A \succ 0$, $\mathbf{b}$, starting point $\mathbf{x}_0$, tolerance $\epsilon > 0$.

1. $\mathbf{g}_0 = A\mathbf{x}_0 - \mathbf{b}, \quad \mathbf{d}_0 = -\mathbf{g}_0$
2. **For** $k = 0, 1, 2, \ldots$:
   - 2.1 Step size: $\alpha_k = \frac{\mathbf{g}_k^T \mathbf{g}_k}{\mathbf{d}_k^T A \mathbf{d}_k}$ [exact line search]
   - 2.2 Update: $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$
   - 2.3 Gradient: $\mathbf{g}_{k+1} = \mathbf{g}_k + \alpha_k A \mathbf{d}_k$ [one mat-vec product!]
   - 2.4 If $\|\mathbf{g}_{k+1}\| < \epsilon$: **stop**
   - 2.5 CG coefficient: $\beta_{k+1} = \frac{\mathbf{g}_{k+1}^T \mathbf{g}_{k+1}}{\mathbf{g}_k^T \mathbf{g}_k}$
   - 2.6 New direction: $\mathbf{d}_{k+1} = -\mathbf{g}_{k+1} + \beta_{k+1} \mathbf{d}_k$

Cost per iteration: One matrix-vector product $A\mathbf{d}_k + O(n)$ vector operations.

Storage: $O(n)$ — just $\mathbf{x}_k$, $\mathbf{g}_k$, $\mathbf{d}_k$, and $A\mathbf{d}_k$.

CG Invariants: Why It Works

Theorem (CG Invariants — exact arithmetic)
After k steps of CG:

1. *Mutual orthogonality of gradients:* $\mathbf{g}_i^T \mathbf{g}_j = 0$ *for* $i \neq j$
2. *A-conjugacy of directions:* $\mathbf{d}_i^T A \mathbf{d}_j = 0$ *for* $i \neq j$
3. *Krylov subspace:*

$$\operatorname{span}\{\mathbf{d}_0, \ldots, \mathbf{d}_k\} = \operatorname{span}\{\mathbf{g}_0, A\mathbf{g}_0, \ldots, A^k \mathbf{g}_0\} = \mathcal{K}_{k+1}(A, \mathbf{g}_0)$$

4. *Optimality:* $\mathbf{x}_{k+1} = \arg\min_{\mathbf{x} \in \mathbf{x}_0 + \mathcal{K}_{k+1}} f(\mathbf{x})$

Property (4) is the most powerful: CG finds the best possible iterate within the Krylov subspace. No method using only matrix-vector products can do better!

Corollary (Finite termination)
Since $\dim \mathcal{K}_k \leq n$ and the subspaces are nested, CG terminates in at most n steps.

Proof of Theorem 5.2 (Expanding Subspace Minimization)

We prove that $\mathbf{x}_k$ minimizes $\phi(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A\mathbf{x} - \mathbf{b}^T\mathbf{x}$ over $\{\mathbf{x} \in \mathbf{x}_0 + \text{span}\{\mathbf{d}_0, \ldots, \mathbf{d}_{k-1}\}\}$.

Step 1: Characterize the minimizer. A point $\tilde{\mathbf{x}}$ minimizes ϕ over $\mathbf{x}_0 + \text{span}\{\mathbf{d}_0, \ldots, \mathbf{d}_{k-1}\}$ if and only if the residual $\mathbf{r}(\tilde{\mathbf{x}}) = \mathbf{b} - A\tilde{\mathbf{x}}$ is orthogonal to every search direction:

$$\mathbf{r}(\tilde{\mathbf{x}})^T \mathbf{d}_i = 0, \qquad i = 0, 1, \ldots, k-1$$

(This is exactly the condition $\nabla\phi(\tilde{\mathbf{x}})^T \mathbf{d}_i = 0$—the gradient is orthogonal to the search subspace.)

Step 2: Base case ($k = 1$). After one CG step, $\mathbf{x}_1 = \mathbf{x}_0 + \alpha_0 \mathbf{d}_0$ where α_0 is chosen to minimize ϕ along $\mathbf{d}_0$. So $\mathbf{r}_1^T \mathbf{d}_0 = 0$ holds by construction.

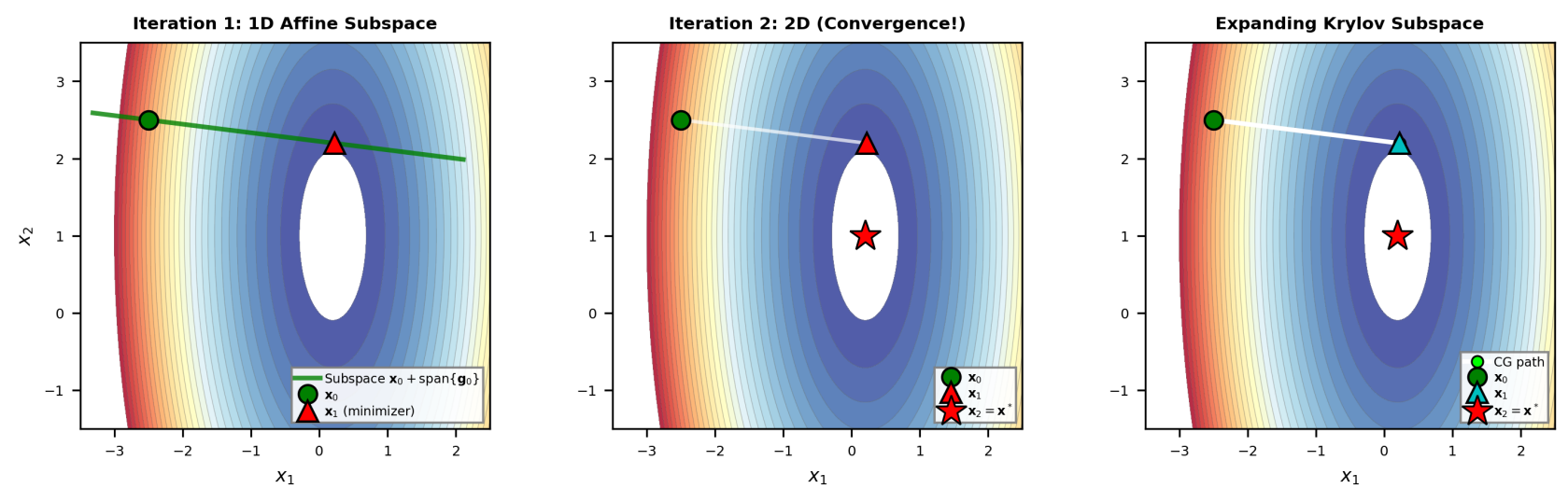
Step 3: Inductive step. Assume $\mathbf{r}_{k-1}^T \mathbf{d}_i = 0$ for $i = 0, \ldots, k-2$. We must show $\mathbf{r}_k^T \mathbf{d}_i = 0$ for $i = 0, \ldots, k-1$.

- For $i = k-1$: The step size α_{k-1} minimizes ϕ along $\mathbf{d}_{k-1}$, so $\mathbf{r}_k^T \mathbf{d}_{k-1} = 0$.
- For $i \leq k-2$: Use $\mathbf{r}_k = \mathbf{r}_{k-1} + \alpha_{k-1} A\mathbf{d}_{k-1}$ to write

$$\mathbf{d}_i^T \mathbf{r}_k = \underbrace{\mathbf{d}_i^T \mathbf{r}_{k-1}}_{=0 \text{ (hypothesis)}} + \alpha_{k-1} \underbrace{\mathbf{d}_i^T A\mathbf{d}_{k-1}}_{=0 \ (A\text{-conjugacy})} = 0 \qquad \checkmark$$

At each step, $\mathbf{x}_k$ is the best possible point in $\mathbf{x}_0 + \text{span}\{\mathbf{d}_0, \ldots, \mathbf{d}_{k-1}\} = \mathbf{x}_0 + \mathcal{K}_k(A, \mathbf{g}_0)$. This is why CG is optimal among Krylov methods.

Visualizing the Expanding Krylov Subspace



Left: At iteration 1, CG minimizes over the 1D affine subspace $\mathbf{x}_0 + \text{span}\{\mathbf{g}_0\}$.

Middle: At iteration 2, the Krylov subspace has expanded to 2D $= \mathbb{R}^n$, so $\mathbf{x}_2 = \mathbf{x}^*$.

Right: The CG path converges in exactly $n = 2$ steps.

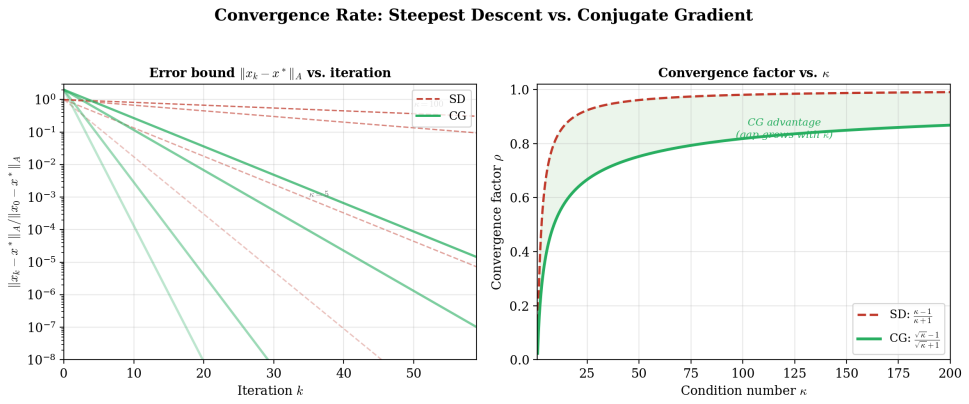
CG Convergence: The $\sqrt{\kappa}$ Effect

Theorem (CG Convergence Bound)

$$\frac{\|\mathbf{x}_k - \mathbf{x}^*\|_A}{\|\mathbf{x}_0 - \mathbf{x}^*\|_A} \leq 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k, \quad \kappa = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}$$

Compare with steepest descent: $\frac{\|\mathbf{x}_{k+1}-\mathbf{x}^*\|_A}{\|\mathbf{x}_k-\mathbf{x}^*\|_A} \leq \frac{\kappa-1}{\kappa+1}$.

CG effectively replaces κ with $\sqrt{\kappa}$!



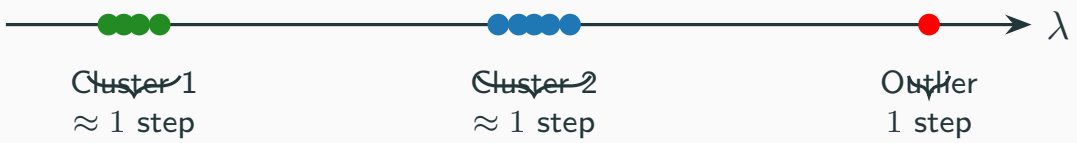
For $\kappa = 100$: SD convergence factor ≈ 0.98 (slow); CG factor ≈ 0.82 (fast).

Why CG Can Be Even Faster: Eigenvalue Clustering

The bound $2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k$ is often **pessimistic**. CG's real power comes from the eigenvalue distribution:

- If A has only r distinct eigenvalues, CG converges in at most $r \leq n$ steps
- If eigenvalues cluster into r groups, CG converges as though dimension were r
- Outlier eigenvalues are "captured" early and stop affecting convergence

3 clusters $\Rightarrow$ CG converges in ≈ 3 steps regardless of n!



This is the basis for preconditioning: find $M \approx A$ so that $M^{-1}A$ has clustered eigenvalues.

Formal Convergence Results (Nocedal & Wright)

These results make precise how the eigenvalue distribution governs CG convergence:

Theorem (Thm 5.4: Distinct eigenvalues $\Rightarrow$ early termination)
If A has only r distinct eigenvalues, then the CG iteration converges in at most r steps.

Proof idea: Construct a polynomial $Q_r(\lambda)$ of degree r with roots at the r distinct eigenvalues and $Q_r(0) = 1$, so that $\max_i |1 + \lambda_i P_{r-1}(\lambda_i)|^2 = 0$.

Theorem (Thm 5.5: Eigenvalue gap bound)

$$\|\mathbf{x}_{k+1} - \mathbf{x}^*\|_A^2 \leq \left(\frac{\lambda_{n-k} - \lambda_1}{\lambda_{n-k} + \lambda_1}\right)^2 \|\mathbf{x}_0 - \mathbf{x}^*\|_A^2$$

This implies CG "captures" the largest eigenvalues first: once the top k eigenvalues are resolved, convergence depends only on the remaining spread λ_{n-k}/λ_1.

Practical implication: If A has m large eigenvalues and the rest are clustered near 1, CG converges well after about $m + 1$ iterations—even if n is enormous.

Preconditioning: Transforming the Spectrum

Key idea: Instead of solving $A\mathbf{x} = \mathbf{b}$, solve the **preconditioned system**:

$$M^{-1}A\mathbf{x} = M^{-1}\mathbf{b}$$

where $M \approx A$ is easy to invert. The eigenvalues of $M^{-1}A$ cluster near 1, dramatically accelerating CG.

Algorithm: Preconditioned CG

Replace the CG algorithm with: at each step, solve $M\mathbf{y}_{k+1} = \mathbf{g}_{k+1}$ (the "preconditioning step"), and use $\mathbf{y}_k$ in place of $\mathbf{g}_k$ in the β and α formulas. Everything else stays the same.

Common preconditioners:

- Incomplete Cholesky: Compute a sparse approximation $\tilde{L}$ to the Cholesky factor of A, set $M = \tilde{L}\tilde{L}^T$
- Jacobi (diagonal): $M = \mathrm{diag}(A)$—cheap but limited
- Multigrid / AMG: For PDE problems, coarser grids provide excellent preconditioners

The art of preconditioning is choosing M that is cheap to invert yet makes $M^{-1}A \approx I$.

The Preconditioned CG Algorithm

Algorithm: Preconditioned Conjugate Gradient
Input: $A \succ 0$, $\mathbf{b}$, preconditioner $M \approx A$ (with M easy to "invert"), starting point $\mathbf{x}_0$, tolerance ϵ.

1. $\mathbf{g}_0 = A\mathbf{x}_0 - \mathbf{b}$, solve $M\mathbf{z}_0 = \mathbf{g}_0$, $\mathbf{d}_0 = -\mathbf{z}_0$
2. **For** $k = 0, 1, 2, \ldots$:
   - 2.1 $\alpha_k = \frac{\mathbf{g}_k^T \mathbf{z}_k}{\mathbf{d}_k^T A \mathbf{d}_k}$ [note: $\mathbf{g}_k^T \mathbf{z}_k$, not $\mathbf{g}_k^T \mathbf{g}_k$]
   - 2.2 $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$
   - 2.3 $\mathbf{g}_{k+1} = \mathbf{g}_k + \alpha_k A \mathbf{d}_k$
   - 2.4 If $\|\mathbf{g}_{k+1}\| < \epsilon$: **stop**
   - 2.5 Solve $M\mathbf{z}_{k+1} = \mathbf{g}_{k+1}$ [the preconditioning step]
   - 2.6 $\beta_{k+1} = \frac{\mathbf{g}_{k+1}^T \mathbf{z}_{k+1}}{\mathbf{g}_k^T \mathbf{z}_k}$
   - 2.7 $\mathbf{d}_{k+1} = -\mathbf{z}_{k+1} + \beta_{k+1} \mathbf{d}_k$

What changed from standard CG? Everywhere we used $\mathbf{g}_k$, we now use $\mathbf{z}_k = M^{-1}\mathbf{g}_k$. The algorithm is mathematically equivalent to running standard CG on the transformed system $\hat{A}\hat{\mathbf{x}} = \hat{\mathbf{b}}$ where $\hat{A} = L^{-1}AL^{-T}$ and $M = LL^T$. But we never form $\hat{A}$—the preconditioned algorithm works entirely in the original coordinates.

Cost per iteration: one mat-vec $A\mathbf{d}_k$ + one preconditioner solve $M\mathbf{z} = \mathbf{g}$ + $O(n)$ vectors. If M is sparse (e.g., incomplete Cholesky), the solve is $O(\text{nnz}(M))$.

Preconditioning Intuition: Why It Works

Without preconditioning: CG convergence depends on $\kappa(A) = \lambda_{\max}/\lambda_{\min}$. For a poorly conditioned system ($\kappa = 10^6$), CG may need $\sim\sqrt{10^6} = 1000$ iterations.

With preconditioning: We effectively solve $M^{-1}A\mathbf{x} = M^{-1}\mathbf{b}$. If $M \approx A$, then $M^{-1}A \approx I$, so $\kappa(M^{-1}A) \approx 1$ and CG converges in a few iterations.

The tradeoff:

Preconditioner	$\kappa(M^{-1}A)$	Cost per solve
None ($M = I$)	$\kappa(A)$	0
Jacobi ($M = \mathrm{diag}(A)$)	$\kappa(A)/c$	$O(n)$
Incomplete Cholesky	$\kappa(A)^{1/2}$ typical	$O(\mathrm{nnz})$
Full Cholesky ($M = A$)	1 (1 iteration!)	$O(n^3)$

The extreme case $M = A$ gives convergence in one step—but then we'd just solve $A\mathbf{x} = \mathbf{b}$ directly. The art is finding M that captures enough of A's structure to cluster the eigenvalues, while being cheap enough that the preconditioner solve doesn't dominate the cost.

For PDE problems, algebraic multigrid (AMG) achieves $O(n)$ total work—each preconditioner solve costs $O(n)$ and CG converges in $O(1)$ iterations independent of n. This is the gold standard for large sparse systems.

Roadmap

Extending CG to General Functions

For nonquadratic $f(\mathbf{x})$, there is no matrix A. The idea: mimic the CG structure using only gradient evaluations.

Algorithm: Nonlinear Conjugate Gradient
Input: Starting point $\mathbf{x}_0$, tolerance $\epsilon > 0$.

$\mathbf{g}_0 = \nabla f(\mathbf{x}_0)$, $\mathbf{d}_0 = -\mathbf{g}_0$.

For $k = 0, 1, 2, \ldots$:

1. Line search: find α_k satisfying Wolfe conditions
2. $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$
3. $\mathbf{g}_{k+1} = \nabla f(\mathbf{x}_{k+1})$
4. If $\|\mathbf{g}_{k+1}\| < \epsilon$: **stop**
5. Compute β_{k+1} [choice of formula $\longrightarrow$ next slide]
6. $\mathbf{d}_{k+1} = -\mathbf{g}_{k+1} + \beta_{k+1} \mathbf{d}_k$

Key difference from linear CG: The step size α_k comes from a Wolfe line search (not the exact formula), and the β formula matters a lot.

The β Formulas: Three Main Variants

On a quadratic with exact line search, all formulas below are equivalent.
For general f, they differ significantly.

Let $\mathbf{y}_k = \mathbf{g}_{k+1} - \mathbf{g}_k$ (gradient difference).

Name	Formula	Key property
Fletcher–Reeves	$\beta_{k+1}^{\text{FR}} = \dfrac{\|\mathbf{g}_{k+1}\|^2}{\|\mathbf{g}_k\|^2}$	Global convergence proof
Polak–Ribière+	$\beta_{k+1}^{\text{PR+}} = \max\left(0,\ \dfrac{\mathbf{g}_{k+1}^T \mathbf{y}_k}{\|\mathbf{g}_k\|^2}\right)$	Automatic restarts
Dai–Yuan	$\beta_{k+1}^{\text{DY}} = \dfrac{\|\mathbf{g}_{k+1}\|^2}{\mathbf{d}_k^T \mathbf{y}_k}$	No restarts needed

Recommendation: Use Polak–Ribière+. When gradients are nearly parallel (flat region), $\mathbf{g}_{k+1}^T \mathbf{y}_k \approx 0$, so $\beta \approx 0$ and the method automatically restarts toward steepest descent. This self-correction is crucial in practice.

Where Do These Formulas Come From?

In linear CG on $A\mathbf{x} = \mathbf{b}$, the CG coefficient is $\beta_{k+1} = \frac{\mathbf{g}_{k+1}^T \mathbf{g}_{k+1}}{\mathbf{g}_k^T \mathbf{g}_k}$. This uses two key identities that hold in exact arithmetic:

$$\mathbf{g}_{k+1}^T \mathbf{g}_k = 0 \qquad \text{and} \qquad \mathbf{d}_k^T A \mathbf{d}_{k-1} = 0$$

For nonlinear f, these identities no longer hold exactly. We can write the linear CG β in several equivalent forms using them. Each form gives a different nonlinear CG method:

Step 1: Fletcher–Reeves: Use $\beta_{k+1} = \frac{\mathbf{g}_{k+1}^T \mathbf{g}_{k+1}}{\mathbf{g}_k^T \mathbf{g}_k}$ directly. This is the simplest generalization—just copy the formula.

Step 2: Polak–Ribière: In linear CG, $\mathbf{g}_{k+1}^T \mathbf{g}_k = 0$, so:

$$\beta_{k+1}^{FR} = \frac{\mathbf{g}_{k+1}^T \mathbf{g}_{k+1}}{\mathbf{g}_k^T \mathbf{g}_k} = \frac{\mathbf{g}_{k+1}^T (\mathbf{g}_{k+1} - \mathbf{g}_k)}{\mathbf{g}_k^T \mathbf{g}_k} = \frac{\mathbf{g}_{k+1}^T \mathbf{y}_k}{\mathbf{g}_k^T \mathbf{g}_k} = \beta_{k+1}^{PR}$$

On quadratics: $\beta^{FR} = \beta^{PR}$. On general functions: they differ because $\mathbf{g}_{k+1}^T \mathbf{g}_k \neq 0$.

Step 3: Hestenes–Stiefel: Similarly, using $\mathbf{d}_k^T A \mathbf{d}_{k-1} = 0$ (from A-conjugacy) gives:

$$\beta_{k+1}^{HS} = \frac{\mathbf{g}_{k+1}^T \mathbf{y}_k}{\mathbf{d}_k^T \mathbf{y}_k}$$

Same numerator as PR, but denominator uses the search direction instead of the gradient.

All three formulas are the same formula written differently, using identities that hold on quadratics. They diverge on general functions because the identities break down.

Why PR+? The Intuition Behind the $\max(0, \cdot)$

The key quantity in PR:
$\mathbf{g}_{k+1}^T \mathbf{y}_k = \mathbf{g}_{k+1}^T(\mathbf{g}_{k+1} - \mathbf{g}_k) = \|\mathbf{g}_{k+1}\|^2 - \mathbf{g}_{k+1}^T \mathbf{g}_k$.

Three regimes:

Case 1: $\mathbf{g}_{k+1}^T \mathbf{g}_k \approx 0$ (gradients nearly orthogonal, like on a quadratic).

Then $\beta^{PR} \approx \|\mathbf{g}_{k+1}\|^2 / \|\mathbf{g}_k\|^2 = \beta^{FR}$. PR and FR agree, and both work well.

Case 2: $\mathbf{g}_{k+1}^T \mathbf{g}_k \approx \|\mathbf{g}_{k+1}\|^2$ (gradients nearly parallel—flat region or tiny step).

Then $\mathbf{g}_{k+1}^T \mathbf{y}_k \approx 0$, so $\beta^{PR} \approx 0$. The next search direction is $\mathbf{d}_{k+1} \approx -\mathbf{g}_{k+1}$: an automatic restart to steepest descent. This is exactly right—if the step was so small that the gradient barely changed, the old conjugate direction is worthless and we should restart.

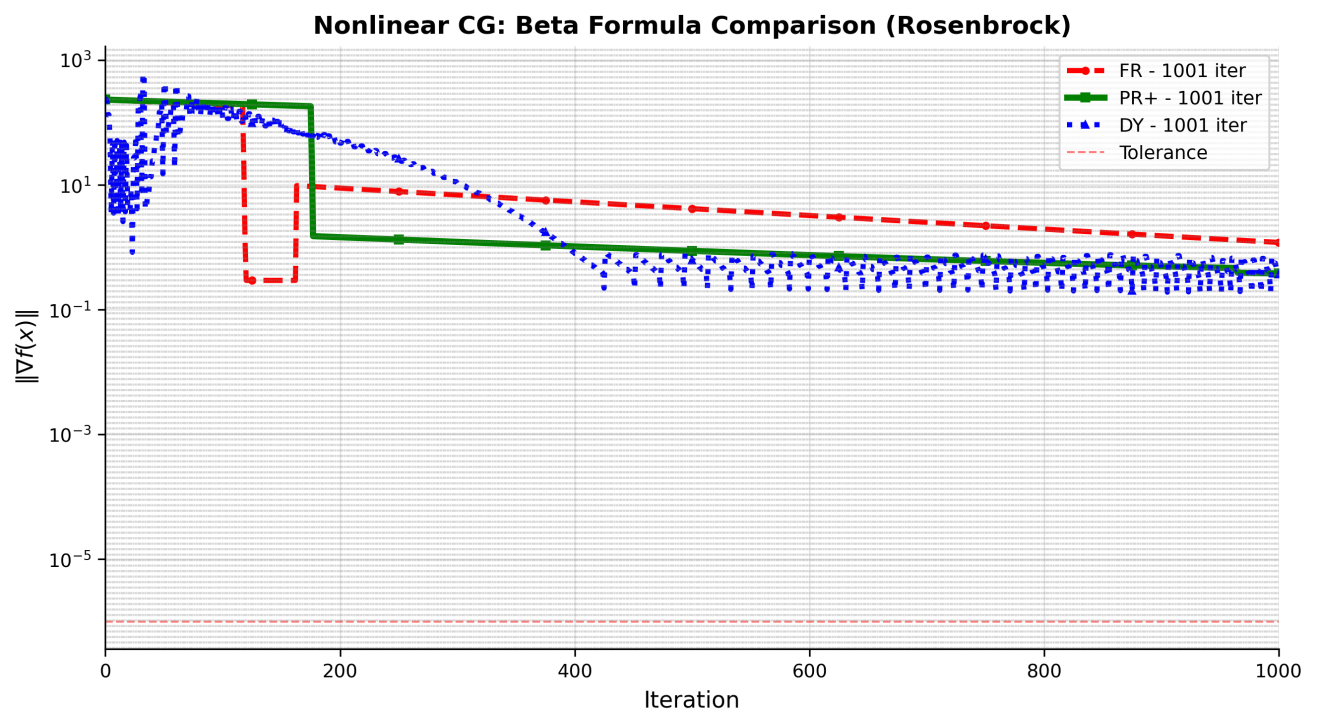
Meanwhile, $\beta^{FR} = \|\mathbf{g}_{k+1}\|^2 / \|\mathbf{g}_k\|^2 \approx 1$. FR blindly continues in the old direction, making no progress. This is why FR gets stuck in long sequences of tiny steps.

Case 3: $\mathbf{g}_{k+1}^T \mathbf{g}_k > \|\mathbf{g}_{k+1}\|^2$ ("uphill" — overshot somehow).

Then $\beta^{PR} < 0$, meaning the conjugate direction would add a component of the old direction pointing uphill. The PR+ fix: set $\beta = 0$ (restart) instead. This prevents the cycling that Powell (1984) showed can happen with negative β.

PR+ combines the best of both worlds: it uses conjugacy information when it's useful ($\beta > 0$), and restarts when it's not ($\beta \leq 0$). The $\max(0, \cdot)$ is not an ad-hoc hack—it's the natural response to the geometry of the gradient sequence.

β Formulas: Convergence Comparison



On the Rosenbrock function, PR+ typically outperforms FR due to its automatic restart behavior. When CG makes poor progress, PR+ detects this and resets to steepest descent.

Convergence Theory for Nonlinear CG

What can we prove? The convergence theory for nonlinear CG is more subtle than the linear case and contains genuine surprises.

Lemma (Lemma 5.6: Descent property of FR)
If Algorithm FR uses strong Wolfe conditions with $0 < c_2 < \frac{1}{2}$, then all search directions $\mathbf{d}_k$ satisfy

$$-\frac{1}{1-c_2} \leq \frac{\nabla f_k^T \mathbf{d}_k}{\|\nabla f_k\|^2} \leq \frac{2c_2 - 1}{1-c_2}$$

In particular, $\nabla f_k^T \mathbf{d}_k < 0$ (descent) for all k.

Theorem (Thm 5.7: Global convergence of FR (Al-Baali))
Under standard assumptions (bounded level sets, Lipschitz gradient), Algorithm FR with strong Wolfe conditions ($0 < c_1 < c_2 < \frac{1}{2}$) satisfies

$$\liminf_{k \to \infty} \|\nabla f_k\| = 0$$

Note: This only gives $\liminf$, not $\lim$—FR may generate long sequences of tiny steps interspersed with good steps.

The Key Tool: Zoutendijk's Theorem

Pronunciation: Zoutendijk = "ZOW-ten-dike" (Dutch, rhymes with "out" + "en" + "dike").

Theorem (Theorem 3.2: Zoutendijk's condition)
Let f be bounded below on $\mathbb{R}^n$ with Lipschitz continuous gradient ($\|\nabla f(\mathbf{x}) - \nabla f(\tilde{\mathbf{x}})\| \leq L\|\mathbf{x} - \tilde{\mathbf{x}}\|$). If the iteration $\mathbf{x}_{k+1} = \mathbf{x}_k + \lambda_k \mathbf{d}_k$ uses descent directions $\mathbf{d}_k$ with step lengths λ_k satisfying the Wolfe conditions, then

$$\boxed{\sum_{k=0}^{\infty} \cos^2 \theta_k \, \|\nabla f_k\|^2 < \infty}$$

where $\cos\theta_k = \frac{-\nabla f_k^T \mathbf{d}_k}{\|\nabla f_k\| \|\mathbf{d}_k\|}$ is the angle between $\mathbf{d}_k$ and $-\nabla f_k$.

Why is this powerful? Since the series converges, either:

- $\|\nabla f_k\| \to 0$ (the gradients converge to zero), **or**
- $\cos\theta_k \to 0$ (the search directions become orthogonal to the gradient—i.e., "bad" directions)

If we can show $\cos\theta_k$ stays bounded away from 0, then $\|\nabla f_k\| \to 0$!

Proof of Zoutendijk's Theorem (Sketch)

Step 1: Lower bound on λ_k: From the curvature Wolfe condition $\nabla f_{k+1}^T \mathbf{d}_k \geq c_2 \nabla f_k^T \mathbf{d}_k$:

$$(\nabla f_{k+1} - \nabla f_k)^T \mathbf{d}_k \geq (c_2 - 1) \nabla f_k^T \mathbf{d}_k$$

By Lipschitz continuity, $(\nabla f_{k+1} - \nabla f_k)^T \mathbf{d}_k \leq \lambda_k L \|\mathbf{d}_k\|^2$. Combining:

$$\lambda_k \geq \frac{c_2 - 1}{L} \cdot \frac{\nabla f_k^T \mathbf{d}_k}{\|\mathbf{d}_k\|^2}$$

Step 2: Substitute into Armijo condition $f_{k+1} \leq f_k + c_1 \lambda_k \nabla f_k^T \mathbf{d}_k$:

$$f_{k+1} \leq f_k - c_1 \frac{1 - c_2}{L} \frac{(\nabla f_k^T \mathbf{d}_k)^2}{\|\mathbf{d}_k\|^2} = f_k - c \cos^2 \theta_k \|\nabla f_k\|^2$$

where $c = c_1(1 - c_2)/L$.

Step 3: Telescope: Sum over $k = 0, \ldots, K$:

$$f_{K+1} \leq f_0 - c \sum_{k=0}^{K} \cos^2 \theta_k \|\nabla f_k\|^2$$

Since f is bounded below, $f_0 - f_{K+1} \leq f_0 - f^*$, so $\sum_{k=0}^{\infty} \cos^2 \theta_k \|\nabla f_k\|^2 < \infty$. $\square$

Proof of Theorem 5.7 (FR Global Convergence)

Strategy: Proof by contradiction using Zoutendijk.

Step 1: Assume for contradiction: $\|\nabla f_k\| \geq \gamma > 0$ for all k sufficiently large.

Step 2: From Lemma 5.6 and Zoutendijk: The descent bounds in Lemma 5.6 imply

$$\frac{1-2c_2}{1-c_2} \leq \cos\theta_k \leq \frac{1}{1-c_2} \cdot \frac{\|\nabla f_k\|}{\|\mathbf{d}_k\|}$$

Substituting into Zoutendijk's condition $\sum \cos^2\theta_k \|\nabla f_k\|^2 < \infty$:

$$\sum_{k=0}^{\infty} \frac{\|\nabla f_k\|^4}{\|\mathbf{d}_k\|^2} < \infty$$

Step 3: Bound $\|\mathbf{d}_k\|$: Using $\mathbf{d}_k = -\nabla f_k + \beta_k^{\text{FR}} \mathbf{d}_{k-1}$ and the strong Wolfe conditions, one shows $\|\mathbf{d}_k\|^2 \leq c_3 \|\nabla f_k\|^4 \sum_{j=0}^{k} \|\nabla f_j\|^{-2}$.

Step 4: Contradiction: Substituting gives $\sum 1/\|\mathbf{d}_k\|^2 < \infty$, but the bound in Step 3 implies $\sum 1/\|\mathbf{d}_k\|^2$ diverges like the harmonic series $\sum 1/k = \infty$.

This contradiction proves $\liminf_{k\to\infty} \|\nabla f_k\| = 0$. □

Key insight: The proof critically uses the two-sided bounds from Lemma 5.6 (which require $c_2 < \frac{1}{2}$). PR does not have these bounds, which is why its convergence theory is weaker.

A Surprising Negative Result

One might expect that the more popular Polak–Ribière method would have even stronger convergence guarantees. In fact, the opposite is true:

Theorem (Thm 5.8: PR can cycle (Powell, 1984))
There exists a twice continuously differentiable $f : \mathbb{R}^3 \to \mathbb{R}$ and a starting point $\mathbf{x}_0$ such that the Polak–Ribière method with an **ideal** *line search produces a sequence of gradients $\{\|\nabla f_k\|\}$ that is bounded away from zero.*

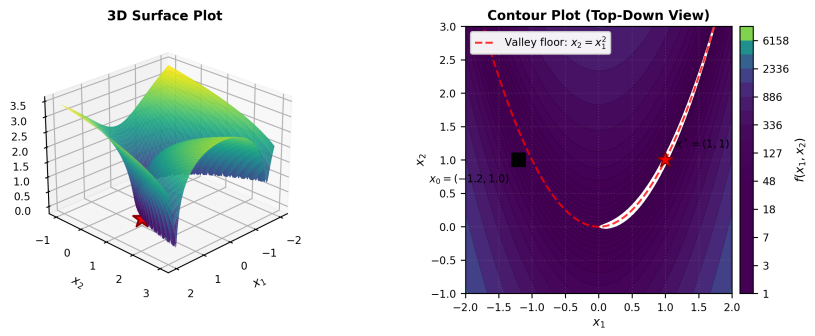
This is remarkable: PR with exact line searches can cycle infinitely without converging, even on smooth functions!

The fix: The PR+ modification $\beta_{k+1}^{+} = \max(0, \beta_{k+1}^{\mathrm{PR}})$ restores global convergence:

- When $\beta^{\mathrm{PR}} < 0$, setting $\beta = 0$ performs a restart to steepest descent
- This prevents the cycling behavior while preserving PR's practical advantages
- With the strong Wolfe conditions, PR+ is provably globally convergent

Takeaway: Algorithm FR has better theory, but PR+ performs better in practice. This is why we recommend PR+.

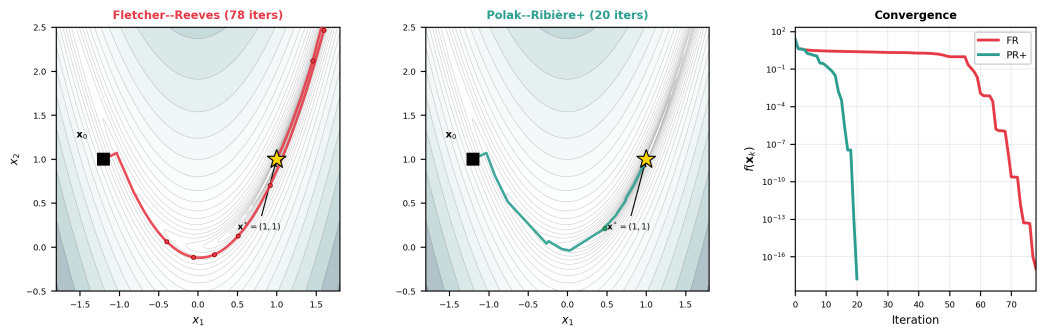
The Rosenbrock Function: Our Benchmark



The Rosenbrock function $f(x_1, x_2) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2$ is the standard test for optimization algorithms.

- The global minimum $\mathbf{x}^* = (1, 1)$ sits at the bottom of a narrow, curved valley.
- Finding the valley is easy; converging along it is hard—the curvature differs by $\approx 10^3$ across vs. along the valley (condition number $\kappa \approx 2508$).
- Generalizes to n dimensions: $f(\mathbf{x}) = \sum_{i=1}^{n-1} \left[100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2\right]$.
- The classic starting point is $\mathbf{x}_0 = (-1.2, 1.0)$, which is inside the valley but far from $\mathbf{x}^*$.

Nonlinear CG on the Rosenbrock Function



Both FR and PR+ navigate the narrow banana valley, but PR+ adapts more effectively. The automatic restart mechanism (setting $\mathbf{d}_k = -\mathbf{g}_k$ when $\beta \leq 0$) prevents the method from persisting in poor search directions.

Python: Nonlinear CG (FR and PR+)

```python
import numpy as np
from scipy.optimize import line_search

f    = lambda x: (1-x[0])**2 + 100*(x[1]-x[0]**2)**2
grad = lambda x: np.array([-2*(1-x[0]) - 400*x[0]*(x[1]-x[0]**2),
                           200*(x[1]-x[0]**2)])

def nonlinear_cg(f, grad, x0, beta='PR+', maxiter=300, tol=1e-8):
    x, g = x0.copy(), grad(x0)
    d = -g.copy()
    for k in range(maxiter):
        res = line_search(f, grad, x, d, c1=1e-4, c2=0.1)
        alpha = res[0] if res[0] else 1e-4
        x = x + alpha * d
        g_new = grad(x)
        if np.linalg.norm(g_new) < tol: break
        if beta == 'FR':
            b = (g_new @ g_new) / (g @ g)
        else:  # PR+
            b = max((g_new @ (g_new - g)) / (g @ g), 0)
        d = -g_new + b * d
        g = g_new
    return x, k+1

x0 = np.array([-1.2, 1.0])
x_fr, it_fr = nonlinear_cg(f, grad, x0, beta='FR')
x_pr, it_pr = nonlinear_cg(f, grad, x0, beta='PR+')
print(f"FR:  {it_fr} iters, f = {f(x_fr):.2e}")
print(f"PR+: {it_pr} iters, f = {f(x_pr):.2e}")
# Typical: FR ~78 iters, PR+ ~20 iters
```

Key difference: PR+ uses `max(..., 0)` to automatically restart when $\beta < 0$.

Python: Nonlinear CG with SciPy (One-Liner)

```python
import numpy as np
from scipy.optimize import minimize

f    = lambda x: (1-x[0])**2 + 100*(x[1]-x[0]**2)**2
grad = lambda x: np.array([-2*(1-x[0]) - 400*x[0]*(x[1]-x[0]**2),
                           200*(x[1]-x[0]**2)])
x0 = np.array([-1.2, 1.0])

# scipy.optimize.minimize with method='CG' uses Polak-Ribiere
res = minimize(f, x0, jac=grad, method='CG')

print(f"Converged: {res.success}")
print(f"Iterations: {res.nit},  Function evals: {res.nfev}")
print(f"Solution:   x* = ({res.x[0]:.6f}, {res.x[1]:.6f})")
print(f"f(x*) = {res.fun:.2e}")

# To track the path, use a callback:
path = [x0.copy()]
minimize(f, x0, jac=grad, method='CG',
         callback=lambda xk: path.append(xk.copy()))

# For n-dimensional Rosenbrock (n=100):
from scipy.optimize import rosen, rosen_der
x0_big = np.zeros(100)
res_big = minimize(rosen, x0_big, jac=rosen_der, method='CG')
print(f"\nn=100 Rosenbrock: {res_big.nit} iters, "
      f"f* = {res_big.fun:.2e}")
```

Note: SciPy's `'CG'` method uses Polak–Ribière with automatic restarts (essentially PR+). There is no option to switch to Fletcher–Reeves.

Roadmap

A Different Approach: Approximate the Hessian

CG's limitation: On nonquadratic problems, CG has only linear convergence and no curvature memory.

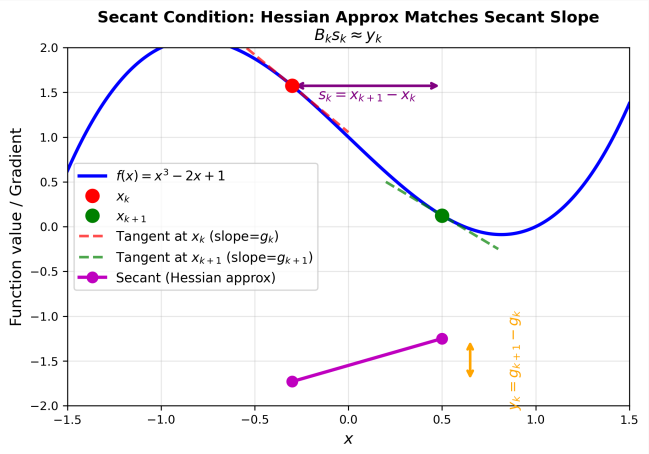
BFGS idea: Build and maintain an approximate Hessian $B_k \approx \nabla^2 f(\mathbf{x}_k)$ using gradient differences.

The Secant Condition

Require the new approximation to match the actual curvature along the most recent step:

$$B_{k+1}\mathbf{s}_k = \mathbf{y}_k$$

where $\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k$ and $\mathbf{y}_k = \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k)$.



Why the Secant Condition? A Taylor's Theorem Argument

The secant condition $B_{k+1}\mathbf{s}_k = \mathbf{y}_k$ is not an arbitrary choice—it follows naturally from calculus.

Step 1: Start with the quadratic model. At $\mathbf{x}_{k+1}$, we build a model:

$$m_{k+1}(\mathbf{p}) = f(\mathbf{x}_{k+1}) + \nabla f(\mathbf{x}_{k+1})^T\mathbf{p} + \tfrac{1}{2}\mathbf{p}^T B_{k+1}\mathbf{p}$$

We want m_{k+1} to be a good local approximation to f.

Step 2: Apply Taylor's theorem to ∇f. By the fundamental theorem of calculus:

$$\nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k) = \left[\int_0^1 \nabla^2 f(\mathbf{x}_k + \tau\alpha_k\mathbf{d}_k)\, d\tau\right](\mathbf{x}_{k+1} - \mathbf{x}_k)$$

i.e., $\mathbf{y}_k = \bar{G}_k\, \mathbf{s}_k$ where $\bar{G}_k$ is the average Hessian along the step.

Step 3: The natural requirement. If B_{k+1} were the true Hessian $\nabla^2 f(\mathbf{x}_{k+1})$, then to first order $B_{k+1}\mathbf{s}_k \approx \mathbf{y}_k$. So imposing $B_{k+1}\mathbf{s}_k = \mathbf{y}_k$ exactly means:

"B_{k+1} acts like the true Hessian along the direction we just traveled."

Step 4: Why only one direction? We have $n(n+1)/2$ unknowns (symmetric matrix) but only n equations from the secant condition. This underdetermination is a feature, not a bug: we use the remaining freedom to stay close to B_k (the minimization principle on the previous slides), preserving curvature information from earlier steps.

The secant condition is the strongest constraint we can impose from one step of gradient information. Any stronger requirement would need second derivatives (Hessian-vector products), which quasi-Newton methods are designed to avoid.

Warm-Up: The Symmetric Rank-One (SR1) Update

Ansatz: $B_{k+1} = B_k + \alpha \mathbf{v}\mathbf{v}^T$ for some $\alpha \in \mathbb{R}$, $\mathbf{v} \in \mathbb{R}^n$.

Step 1: Apply the secant condition:

$$(B_k + \alpha \mathbf{v}\mathbf{v}^T)\mathbf{s}_k = \mathbf{y}_k \quad \Longrightarrow \quad \alpha(\mathbf{v}^T\mathbf{s}_k)\mathbf{v} = \mathbf{y}_k - B_k\mathbf{s}_k$$

Step 2: The RHS is proportional to $\mathbf{v}$:

$$\mathbf{v} = \beta(\mathbf{y}_k - B_k\mathbf{s}_k) \quad \text{for some } \beta \neq 0$$

Step 3: Substitute back and solve: $\alpha\beta^2[(\mathbf{y}_k - B_k\mathbf{s}_k)^T\mathbf{s}_k] = 1$

SR1 formula:

$$\boxed{B_{k+1}^{\text{SR1}} = B_k + \frac{(\mathbf{y}_k - B_k\mathbf{s}_k)(\mathbf{y}_k - B_k\mathbf{s}_k)^T}{(\mathbf{y}_k - B_k\mathbf{s}_k)^T\mathbf{s}_k}}$$

Problem: The SR1 update does **not** preserve positive definiteness! If $B_k \succ 0$, B_{k+1}^{SR1} may be indefinite. Can we do better with a rank-2 correction?

Deriving the BFGS Update: Step by Step

Ansatz: $B_{k+1} = B_k + \alpha \mathbf{u}\mathbf{u}^T + \beta \mathbf{v}\mathbf{v}^T$ (rank-2 symmetric correction).

Step 1: **Apply the secant condition** $B_{k+1}\mathbf{s}_k = \mathbf{y}_k$:

$$(B_k + \alpha \mathbf{u}\mathbf{u}^T + \beta \mathbf{v}\mathbf{v}^T)\mathbf{s}_k = \mathbf{y}_k$$

$$\implies \quad \alpha(\mathbf{u}^T\mathbf{s}_k)\mathbf{u} + \beta(\mathbf{v}^T\mathbf{s}_k)\mathbf{v} = \mathbf{y}_k - B_k\mathbf{s}_k$$

Step 2: **Match the two terms separately** (a natural splitting):

$$\alpha(\mathbf{u}^T\mathbf{s}_k)\mathbf{u} = \mathbf{y}_k, \qquad \beta(\mathbf{v}^T\mathbf{s}_k)\mathbf{v} = -B_k\mathbf{s}_k$$

Step 3: **From the first equation:** $\mathbf{u} \| \mathbf{y}_k$. Choose $\mathbf{u} = \mathbf{y}_k$. Then:

$$\alpha(\mathbf{y}_k^T\mathbf{s}_k)\mathbf{y}_k = \mathbf{y}_k \quad \implies \quad \alpha = \frac{1}{\mathbf{y}_k^T\mathbf{s}_k}$$

Step 4: **From the second equation:** $\mathbf{v} \| B_k\mathbf{s}_k$. Choose $\mathbf{v} = B_k\mathbf{s}_k$.
Then:

$$\beta(\mathbf{s}_k^T B_k\mathbf{s}_k)\, B_k\mathbf{s}_k = -B_k\mathbf{s}_k \quad \implies \quad \beta = -\frac{1}{\mathbf{s}_k^T B_k\mathbf{s}_k}$$

The BFGS Update Formula

Substituting $\mathbf{u} = \mathbf{y}_k$, $\alpha = \frac{1}{\mathbf{y}_k^T \mathbf{s}_k}$, $\mathbf{v} = B_k \mathbf{s}_k$, $\beta = -\frac{1}{\mathbf{s}_k^T B_k \mathbf{s}_k}$:

BFGS formula (Broyden–Fletcher–Goldfarb–Shanno, 1970):

$$B_{k+1} = B_k + \frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} - \frac{B_k \mathbf{s}_k \mathbf{s}_k^T B_k}{\mathbf{s}_k^T B_k \mathbf{s}_k}$$

B_k
Current approx.

$-$

$\frac{B_k \mathbf{s}_k \mathbf{s}_k^T B_k}{\mathbf{s}_k^T B_k \mathbf{s}_k}$
"Forget" direction $\mathbf{s}_k$

$+$

$\frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k}$
"Learn" new curvature

Interpretation:

1. **Subtraction:** Remove the component of B_k along $\mathbf{s}_k$—"forgets" old curvature in the step direction.
2. **Addition:** Insert new curvature from $\mathbf{y}_k$—"learns" the true curvature along $\mathbf{s}_k$.

B_{k+1} agrees with B_k in all directions $\perp \mathbf{s}_k$, but matches the true curvature along $\mathbf{s}_k$.

An Alternative Derivation: The Frobenius Norm Approach

Our ansatz derivation assumed a rank-2 form $B_{k+1} = B_k + \alpha \mathbf{u}\mathbf{u}^T + \beta \mathbf{v}\mathbf{v}^T$ and matched terms. But why rank 2? Nocedal & Wright (§6.1) answer this with an optimization principle: find the closest symmetric matrix to B_k satisfying the secant condition.

The Frobenius norm. For a matrix $A \in \mathbb{R}^{n \times n}$, the Frobenius norm is:

$$\|A\|_F = \sqrt{\sum_{i=1}^{n} \sum_{j=1}^{n} a_{ij}^2} = \sqrt{\text{trace}(A^T A)}$$

Think of it as "flatten A into a vector of n^2 entries, then take the Euclidean norm." It measures the total magnitude of all entries. The minimization $\min_B \|B - B_k\|_F$ subject to constraints finds the matrix B that changes B_k as little as possible (in a sum-of-squares sense) while satisfying the constraints.

The weighted Frobenius norm. A bare $\|\cdot\|_F$ has a problem: it depends on how we scale the variables. If we change coordinates $\mathbf{x} \to T\mathbf{x}$, the Hessian transforms as $B \to T^{-T} B T^{-1}$, and "closeness" in the old coordinates differs from closeness in the new ones. To fix this, we use:

$$\|A\|_W = \|W^{1/2} A W^{1/2}\|_F$$

with $W = \bar{G}_k^{-1}$, the inverse of the average Hessian $\bar{G}_k = \int_0^1 \nabla^2 f(\mathbf{x}_k + \tau \alpha_k \mathbf{d}_k)\, d\tau$. This makes the solution scale-invariant—it doesn't matter what units we use for $\mathbf{x}$.

Any W satisfying $W\mathbf{y}_k = \mathbf{s}_k$ gives the same formulas. The average Hessian is the natural choice because $\mathbf{y}_k = \bar{G}_k \mathbf{s}_k$ (by the fundamental theorem of calculus), so $\bar{G}_k^{-1}\mathbf{y}_k = \mathbf{s}_k$ holds automatically.

BFGS via Minimization: DFP and BFGS as Dual Problems

With the weighted Frobenius norm in hand, we can state the two minimization problems:

For B_{k+1} (DFP derivation):

$$\min_B \|B - B_k\|_W \quad \text{s.t. } B = B^T, \ B\mathbf{s}_k = \mathbf{y}_k$$

For H_{k+1} (BFGS derivation):

$$\min_H \|H - H_k\|_W \quad \text{s.t. } H = H^T, \ H\mathbf{y}_k = \mathbf{s}_k$$

These are dual problems—swap $B \leftrightarrow H$ and $\mathbf{s}_k \leftrightarrow \mathbf{y}_k$. Solving the first gives DFP; solving the second gives BFGS. Both have unique solutions. But note: "minimize over B, then invert" $\neq$ "minimize over H directly." Closeness in B-space is not closeness in H-space, so DFP $\neq$ BFGS in general.

How the two derivations connect:

- The minimization view answers: "Why this rank-2 update and not some other?" Answer: it is the closest to B_k (or H_k) that satisfies the secant condition. The rank-2 structure emerges from the Lagrange multiplier solution (next slide).
- Our ansatz derivation shows how to compute the formula by matching coefficients.
- Both give the same result—the BFGS formula is the unique solution to the H-minimization problem.

On a quadratic with exact line search, DFP and BFGS generate the same iterates (Thm 6.4). On nonquadratic problems, they differ—and BFGS wins due to its superior self-correcting properties.

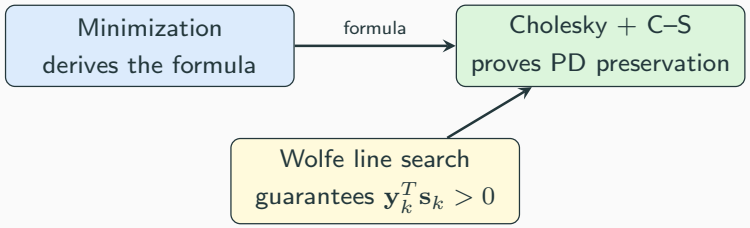
The Minimization View and Positive Definiteness

An important subtlety: the minimization problem $\min_B \|B - B_k\|_W$ subject to $B = B^T$, $B\mathbf{s}_k = \mathbf{y}_k$ does not explicitly require $B_{k+1} \succ 0$. So why does BFGS preserve PD?

The connection is indirect:

1. The minimization tells us what formula to use (it uniquely determines the update).
2. Our Cholesky + Cauchy–Schwarz proof (Slide 39) shows that this particular formula happens to preserve PD whenever $\mathbf{y}_k^T \mathbf{s}_k > 0$.
3. The Wolfe line search (Slide 40) guarantees $\mathbf{y}_k^T \mathbf{s}_k > 0$.

These are three independent facts that work together:



Nocedal & Wright give an alternative PD proof directly from the H_k update (p. 141): for any $\mathbf{z} \neq 0$, write $\mathbf{z}^T H_{k+1} \mathbf{z} = \mathbf{w}^T H_k \mathbf{w} + \rho_k (\mathbf{z}^T \mathbf{s}_k)^2 \geq 0$ where $\mathbf{w} = \mathbf{z} - \rho_k \mathbf{y}_k (\mathbf{s}_k^T \mathbf{z})$. This works because $H_k \succ 0$ and $\rho_k > 0$, without needing Cholesky.

Solving the Minimization Problems

How do we actually solve $\min_H \|H - H_k\|_W$ subject to $H = H^T$, $H\mathbf{y}_k = \mathbf{s}_k$?

Step 1: Change variables. Define $\hat{H} = W^{1/2} H W^{1/2}$ and $\hat{H}_k = W^{1/2} H_k W^{1/2}$. Then $\|H - H_k\|_W = \|\hat{H} - \hat{H}_k\|_F$ (just the ordinary Frobenius norm). The constraint $H\mathbf{y}_k = \mathbf{s}_k$ becomes $\hat{H}\hat{\mathbf{y}}_k = \hat{\mathbf{s}}_k$ where $\hat{\mathbf{y}}_k = W^{-1/2}\mathbf{y}_k$, $\hat{\mathbf{s}}_k = W^{1/2}\mathbf{s}_k$.

Step 2: Write the Lagrangian. We minimize $\frac{1}{2}\|\hat{H} - \hat{H}_k\|_F^2$ over symmetric $\hat{H}$ subject to $\hat{H}\hat{\mathbf{y}}_k = \hat{\mathbf{s}}_k$. Using a Lagrange multiplier vector $\boldsymbol{\lambda} \in \mathbb{R}^n$:

$$\mathcal{L}(\hat{H}, \boldsymbol{\lambda}) = \tfrac{1}{2}\|\hat{H} - \hat{H}_k\|_F^2 - \boldsymbol{\lambda}^T(\hat{H}\hat{\mathbf{y}}_k - \hat{\mathbf{s}}_k)$$

Step 3: Stationarity. Setting $\frac{\partial \mathcal{L}}{\partial \hat{H}} = 0$ (and enforcing symmetry) gives:

$$\hat{H} = \hat{H}_k + \frac{1}{2}(\boldsymbol{\lambda}\hat{\mathbf{y}}_k^T + \hat{\mathbf{y}}_k\boldsymbol{\lambda}^T)$$

The solution differs from $\hat{H}_k$ by a symmetric rank-2 correction in the directions $\boldsymbol{\lambda}$ and $\hat{\mathbf{y}}_k$.

Step 4: Solve for $\boldsymbol{\lambda}$. Substitute into $\hat{H}\hat{\mathbf{y}}_k = \hat{\mathbf{s}}_k$ and solve the resulting linear system. This determines $\boldsymbol{\lambda}$ uniquely.

Step 5: Transform back. Apply $H = W^{-1/2}\hat{H}W^{-1/2}$ and use $W\mathbf{y}_k = \mathbf{s}_k$ to simplify. The result is exactly the BFGS formula (eq. 6.17).

The Lagrange multiplier approach automatically produces a rank-2 update—we don't need to guess the ansatz! The constraint $H\mathbf{y}_k = \mathbf{s}_k$ has n equations, but symmetry of H reduces the effective degrees of freedom, yielding a rank-2 correction.

Initialization: The First Step Is a Gradient Step

How do we start? We need an initial approximation B_0 (or $H_0 = B_0^{-1}$).

Standard choice: $H_0 = I$ (i.e., $B_0 = I$).

Then the first search direction is:

$$\mathbf{d}_0 = -H_0 \nabla f(\mathbf{x}_0) = -\nabla f(\mathbf{x}_0)$$

This is exactly steepest descent! So **yes, the first step of BFGS is always a gradient step**.

After the first step, BFGS has one curvature pair $(\mathbf{s}_0, \mathbf{y}_0)$ and builds $H_1 \neq I$. From iteration 2 onward, the search directions incorporate curvature and are no longer gradient directions.

Practical improvement (initial scaling heuristic): After computing the first pair $(\mathbf{s}_0, \mathbf{y}_0)$, reset:

$$H_0 \leftarrow \frac{\mathbf{y}_0^T \mathbf{s}_0}{\mathbf{y}_0^T \mathbf{y}_0} I$$

This scales H_0 to approximate the magnitude of the true inverse Hessian, giving a much better second step. L-BFGS does this automatically at every iteration.

BFGS Preserves Positive Definiteness

Theorem
If $B_k \succ 0$ and $\mathbf{y}_k^T \mathbf{s}_k > 0$, then $B_{k+1}^{\text{BFGS}} \succ 0$.

Proof. Since $B_k \succ 0$, it has a Cholesky factorization $B_k = L_k L_k^T$.
Define the vectors

$$\mathbf{u} = L_k^T \mathbf{z}, \qquad \mathbf{v} = L_k^T \mathbf{s}_k.$$

Step 1: Substitute the BFGS update $B_{k+1} = B_k + \frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} - \frac{B_k \mathbf{s}_k \mathbf{s}_k^T B_k}{\mathbf{s}_k^T B_k \mathbf{s}_k}$.
For any $\mathbf{z} \neq \mathbf{0}$:

$$\mathbf{z}^T B_{k+1} \mathbf{z} = \underbrace{\mathbf{z}^T B_k \mathbf{z}}_{\|\mathbf{u}\|^2} - \frac{(\mathbf{z}^T B_k \mathbf{s}_k)^2}{\mathbf{s}_k^T B_k \mathbf{s}_k} + \frac{(\mathbf{z}^T \mathbf{y}_k)^2}{\mathbf{y}_k^T \mathbf{s}_k}$$

Step 2: Rewrite in terms of $\mathbf{u}$ and $\mathbf{v}$. Since $\mathbf{z}^T B_k \mathbf{s}_k = \mathbf{z}^T L_k L_k^T \mathbf{s}_k = \mathbf{u}^T \mathbf{v}$:

$$\mathbf{z}^T B_{k+1} \mathbf{z} = \|\mathbf{u}\|^2 - \frac{(\mathbf{u}^T \mathbf{v})^2}{\|\mathbf{v}\|^2} + \frac{(\mathbf{z}^T \mathbf{y}_k)^2}{\mathbf{y}_k^T \mathbf{s}_k}$$

Step 3: Apply standard Cauchy–Schwarz $(\mathbf{u}^T \mathbf{v})^2 \leq \|\mathbf{u}\|^2 \|\mathbf{v}\|^2$ to the first two terms:

$$\|\mathbf{u}\|^2 - \frac{(\mathbf{u}^T \mathbf{v})^2}{\|\mathbf{v}\|^2} \geq \|\mathbf{u}\|^2 - \|\mathbf{u}\|^2 = 0$$

The third term $\frac{(\mathbf{z}^T \mathbf{y}_k)^2}{\mathbf{y}_k^T \mathbf{s}_k} \geq 0$ since $\mathbf{y}_k^T \mathbf{s}_k > 0$. So $\mathbf{z}^T B_{k+1} \mathbf{z} \geq 0$.

Step 4: **Strict positivity:** Both terms vanish iff $\mathbf{u} \| \mathbf{v}$ (i.e., $\mathbf{z} \| \mathbf{s}_k$) *and* $\mathbf{z} \perp \mathbf{y}_k$. But then $\mathbf{s}_k \perp \mathbf{y}_k$, contradicting $\mathbf{y}_k^T \mathbf{s}_k > 0$. Hence $\mathbf{z}^T B_{k+1} \mathbf{z} > 0$. □

The Wolfe line search guarantees $\mathbf{y}_k^T \mathbf{s}_k > 0$, so BFGS always stays PD!

Why the Wolfe Conditions Guarantee $\mathbf{y}_k^T \mathbf{s}_k > 0$

The PD preservation theorem requires $\mathbf{y}_k^T \mathbf{s}_k > 0$. This is not an extra assumption—it follows automatically from the Wolfe conditions.

Lemma
If α_k satisfies the (weak) Wolfe conditions with $0 < c_1 < c_2 < 1$, then $\mathbf{y}_k^T \mathbf{s}_k > 0$.

Proof. The curvature condition (second Wolfe condition) states:

$$\nabla f(\mathbf{x}_{k+1})^T \mathbf{d}_k \geq c_2 \, \nabla f(\mathbf{x}_k)^T \mathbf{d}_k$$

Since $\mathbf{s}_k = \alpha_k \mathbf{d}_k$ with $\alpha_k > 0$, multiplying both sides by α_k:

$$\nabla f(\mathbf{x}_{k+1})^T \mathbf{s}_k \geq c_2 \, \nabla f(\mathbf{x}_k)^T \mathbf{s}_k$$

Now subtract $\nabla f(\mathbf{x}_k)^T \mathbf{s}_k$ from both sides:

$$\mathbf{y}_k^T \mathbf{s}_k = \big(\nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k)\big)^T \mathbf{s}_k \geq (c_2 - 1) \, \nabla f(\mathbf{x}_k)^T \mathbf{s}_k$$

Since $\mathbf{d}_k$ is a descent direction, $\nabla f(\mathbf{x}_k)^T \mathbf{d}_k < 0$, so $\nabla f(\mathbf{x}_k)^T \mathbf{s}_k < 0$. Since $c_2 < 1$, we have $(c_2 - 1) < 0$, and therefore:

$$\mathbf{y}_k^T \mathbf{s}_k \geq \underbrace{(c_2 - 1)}_{<0} \underbrace{\nabla f(\mathbf{x}_k)^T \mathbf{s}_k}_{<0} > 0. \quad \square$$

This is why the Wolfe line search is essential for quasi-Newton methods—it is the mechanism that keeps $B_k \succ 0$ at every iteration.

Line Search in Practice: How Wolfe Conditions Are Enforced

The theory requires step sizes α_k satisfying the Wolfe conditions. But how does a line search algorithm actually find such an α_k?

The Wolfe Conditions (recap)
Given descent direction $\mathbf{d}_k$, find $\alpha > 0$ satisfying:

1. Sufficient decrease (Armijo):
   $f(\mathbf{x}_k + \alpha \mathbf{d}_k) \leq f(\mathbf{x}_k) + c_1 \alpha \nabla f(\mathbf{x}_k)^T \mathbf{d}_k$ $[c_1 = 10^{-4}]$
2. Curvature condition: $\nabla f(\mathbf{x}_k + \alpha \mathbf{d}_k)^T \mathbf{d}_k \geq c_2 \nabla f(\mathbf{x}_k)^T \mathbf{d}_k$ $[c_2 = 0.9$ for BFGS]

The two-phase algorithm (Nocedal & Wright, Algorithm 3.5–3.6):

Step 1: Bracketing phase. Start with $\alpha = 1$ (natural for quasi-Newton). If Armijo fails, we overshot—shrink. If Armijo holds but curvature fails, we haven't gone far enough—expand. Continue until we find an interval $[\alpha_{\text{lo}}, \alpha_{\text{hi}}]$ guaranteed to contain a point satisfying both conditions.

Step 2: Zoom phase. Bisect (or interpolate) within $[\alpha_{\text{lo}}, \alpha_{\text{hi}}]$ to find the actual Wolfe point. Use cubic interpolation of f and ∇f values for fast convergence.

Why $\alpha = 1$ first? For quasi-Newton methods, $\mathbf{d}_k \approx -(\nabla^2 f)^{-1} \nabla f_k$, so the Newton step corresponds to $\alpha = 1$. Near the solution, $\alpha_k = 1$ is accepted without any backtracking—this is what enables superlinear convergence.

Line Search: The Geometry

What can go wrong with a simple backtracking line search?

A backtracking line search (Armijo only) starts at $\alpha = 1$ and repeatedly multiplies by $\tau \in (0, 1)$ until sufficient decrease holds. This is simple but has a critical flaw for quasi-Newton methods: it doesn't enforce the curvature condition.

Without the curvature condition:

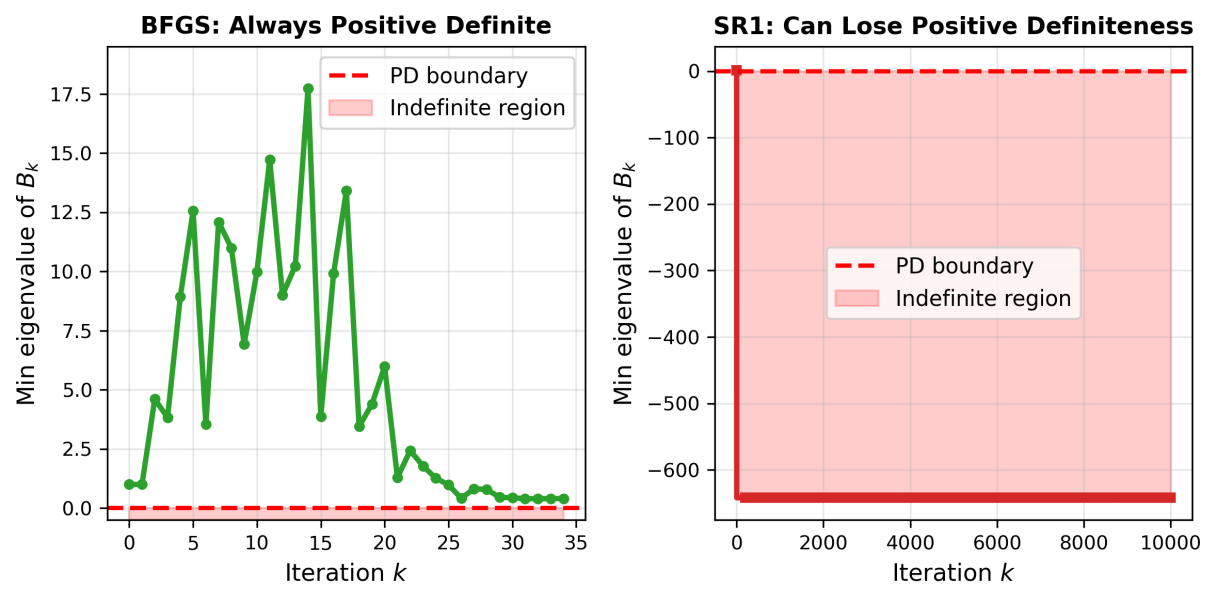
- The step size can be too small—we stop at the first α satisfying Armijo, which may be far from the minimum along the ray
- We get $\mathbf{y}_k^T \mathbf{s}_k \leq 0$ (the curvature pair is invalid)
- The BFGS update may destroy positive definiteness
- The Hessian approximation degrades, and self-correction fails

With the full Wolfe conditions:

- The step size is large enough to capture meaningful curvature
- $\mathbf{y}_k^T \mathbf{s}_k > 0$ is guaranteed, preserving PD
- $\alpha_k = 1$ is accepted near the solution (enabling superlinear convergence)
- For BFGS, use $c_2 = 0.9$ (loose). For nonlinear CG, use $c_2 = 0.1$ (tight—CG is more sensitive to the line search quality)

Never use plain Armijo backtracking with BFGS. The extra cost of enforcing the curvature condition is small (one additional gradient evaluation) and the benefit is enormous.

Positive Definiteness: Visualization



Left (BFGS): Via Cholesky $B_k = LL^T$, standard Cauchy–Schwarz gives $\|L^T\mathbf{z}\|^2 - \frac{((L^T\mathbf{z})^T(L^T\mathbf{s}_k))^2}{\|L^T\mathbf{s}_k\|^2} \geq 0$, while $\frac{(\mathbf{z}^T\mathbf{y}_k)^2}{\mathbf{y}_k^T\mathbf{s}_k} \geq 0$ ensures $B_{k+1} \succ 0$.

Right (SR1): No such guarantee—SR1 can lose positive definiteness, leading to non-descent directions.

Why Store the Inverse?

The cost comparison:

Strategy 1: Store B_k
Update $B_k \to B_{k+1}$: $O(n^2)$
Solve $B_{k+1}\mathbf{d} = -\nabla f$: $O(n^3)$

Strategy 2: Store $H_k = B_k^{-1}$
Update $H_k \to H_{k+1}$: $O(n^2)$
Compute $\mathbf{d} = -H_{k+1}\nabla f$: $O(n^2)$

Strategy 2 is strictly cheaper! But can we derive an update for $H_k = B_k^{-1}$ directly from the BFGS update for B_k?

Yes—using the **Sherman–Morrison–Woodbury formula**.

Tool: The Sherman–Morrison Formula

Theorem (Sherman–Morrison)
If $A \in \mathbb{R}^{n \times n}$ is invertible and $1 + \mathbf{v}^T A^{-1} \mathbf{u} \neq 0$, then:

$$(A + \mathbf{u}\mathbf{v}^T)^{-1} = A^{-1} - \frac{A^{-1}\mathbf{u}\mathbf{v}^T A^{-1}}{1 + \mathbf{v}^T A^{-1}\mathbf{u}}$$

Why is this powerful?

- A rank-1 correction to A needs only a rank-1 correction to A^{-1}
- Cost: $O(n^2)$ (two matrix-vector products + one outer product)
- For rank-2 corrections (like BFGS): apply Sherman–Morrison twice

Intuition: Why Sherman–Morrison Works

The formula looks magical at first, but the idea is natural once you think about what a rank-1 perturbation does.

The key insight: $A + \mathbf{u}\mathbf{v}^T$ changes A only in one direction. For any vector $\mathbf{x}$:

$$(A + \mathbf{u}\mathbf{v}^T)\mathbf{x} = A\mathbf{x} + \mathbf{u}\underbrace{(\mathbf{v}^T\mathbf{x})}_{\text{scalar}}$$

The perturbation adds a multiple of $\mathbf{u}$ scaled by how much $\mathbf{x}$ aligns with $\mathbf{v}$. If $\mathbf{x} \perp \mathbf{v}$, the perturbation does nothing—the matrix acts exactly like A.

So to invert the perturbed system $(A + \mathbf{u}\mathbf{v}^T)\mathbf{x} = \mathbf{b}$:

1. Start with the "naive" solution $\mathbf{x}_0 = A^{-1}\mathbf{b}$ (ignoring the perturbation).
2. This overshoots by $\mathbf{u}(\mathbf{v}^T A^{-1}\mathbf{b})$—the error is along $A^{-1}\mathbf{u}$.
3. Apply a scalar correction to cancel that error: subtract $\frac{\mathbf{v}^T A^{-1}\mathbf{b}}{1+\mathbf{v}^T A^{-1}\mathbf{u}}\, A^{-1}\mathbf{u}$.
4. The denominator $1 + \mathbf{v}^T A^{-1}\mathbf{u}$ accounts for the feedback loop (the correction itself gets perturbed).

Sherman–Morrison is just "solve ignoring the perturbation, then correct for the one direction it affects." It is a rank-1 perturbation, so it needs a rank-1 correction.

Proof of the Sherman–Morrison Formula

Goal: Show $(A + \mathbf{u}\mathbf{v}^T)^{-1} = A^{-1} - \frac{A^{-1}\mathbf{u}\mathbf{v}^T A^{-1}}{1+\mathbf{v}^T A^{-1}\mathbf{u}}$.

Strategy: Multiply $(A + \mathbf{u}\mathbf{v}^T)$ by the proposed inverse and verify $= I$.

Let $C = A^{-1} - \frac{A^{-1}\mathbf{u}\mathbf{v}^T A^{-1}}{1+\mathbf{v}^T A^{-1}\mathbf{u}}$. Compute:

$$
\begin{aligned}
(A + \mathbf{u}\mathbf{v}^T)C &= AC + \mathbf{u}\mathbf{v}^T C \\
&= \underbrace{AA^{-1}}_{I} - \frac{AA^{-1}\mathbf{u}\mathbf{v}^T A^{-1}}{1 + \mathbf{v}^T A^{-1}\mathbf{u}} + \mathbf{u}\mathbf{v}^T A^{-1} - \frac{\mathbf{u}\mathbf{v}^T A^{-1}\mathbf{u}\mathbf{v}^T A^{-1}}{1 + \mathbf{v}^T A^{-1}\mathbf{u}} \\
&= I - \frac{\mathbf{u}\mathbf{v}^T A^{-1}}{1 + \mathbf{v}^T A^{-1}\mathbf{u}} + \mathbf{u}\mathbf{v}^T A^{-1} - \frac{(\mathbf{v}^T A^{-1}\mathbf{u})\,\mathbf{u}\mathbf{v}^T A^{-1}}{1 + \mathbf{v}^T A^{-1}\mathbf{u}}
\end{aligned}
$$

Factor out $\mathbf{u}\mathbf{v}^T A^{-1}$ from the last three terms:

$$
\begin{aligned}
&= I + \mathbf{u}\mathbf{v}^T A^{-1}\left[\frac{-1}{1 + \mathbf{v}^T A^{-1}\mathbf{u}} + 1 - \frac{\mathbf{v}^T A^{-1}\mathbf{u}}{1 + \mathbf{v}^T A^{-1}\mathbf{u}}\right] \\
&= I + \mathbf{u}\mathbf{v}^T A^{-1}\left[\frac{-1 + (1 + \mathbf{v}^T A^{-1}\mathbf{u}) - \mathbf{v}^T A^{-1}\mathbf{u}}{1 + \mathbf{v}^T A^{-1}\mathbf{u}}\right] = I + \mathbf{u}\mathbf{v}^T A^{-1} \cdot \frac{0}{1 + \mathbf{v}^T A^{-1}\mathbf{u}} = I. \qquad \square
\end{aligned}
$$

Deriving the BFGS Inverse Update

Goal: Given $B_{k+1} = B_k + \frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} - \frac{B_k \mathbf{s}_k \mathbf{s}_k^T B_k}{\mathbf{s}_k^T B_k \mathbf{s}_k}$, find $H_{k+1} = B_{k+1}^{-1}$.

Step 1: First rank-1 update (positive term):

Write $\tilde{B} = B_k + \frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} = B_k + \mathbf{u}\mathbf{v}^T$ with $\mathbf{u} = \frac{\mathbf{y}_k}{\mathbf{y}_k^T \mathbf{s}_k}$, $\mathbf{v} = \mathbf{y}_k$.

Apply Sherman–Morrison with $A = B_k$, $A^{-1} = H_k$:

$$\tilde{B}^{-1} = H_k - \frac{H_k \mathbf{y}_k \mathbf{y}_k^T H_k}{\mathbf{y}_k^T \mathbf{s}_k + \mathbf{y}_k^T H_k \mathbf{y}_k}$$

Step 2: Second rank-1 update (negative term):

$B_{k+1} = \tilde{B} - \frac{B_k \mathbf{s}_k \mathbf{s}_k^T B_k}{\mathbf{s}_k^T B_k \mathbf{s}_k}$: another rank-1 correction to $\tilde{B}$.

Apply Sherman–Morrison again to $\tilde{B}$.

Step 3: After careful algebra (using $H_k \mathbf{y}_k = H_k B_k \mathbf{s}_k$ and $B_k \mathbf{s}_k = B_k H_k \mathbf{y}_k$; see Nocedal & Wright §6.1), the result simplifies to:

Let $\rho_k = \frac{1}{\mathbf{y}_k^T \mathbf{s}_k}$:

$$\boxed{H_{k+1} = (I - \rho_k \mathbf{s}_k \mathbf{y}_k^T)\, H_k\, (I - \rho_k \mathbf{y}_k \mathbf{s}_k^T) + \rho_k \mathbf{s}_k \mathbf{s}_k^T}$$

The BFGS Inverse Update: Compact Form

Denoting $V_k = I - \rho_k \mathbf{y}_k \mathbf{s}_k^T$ where $\rho_k = \frac{1}{\mathbf{y}_k^T \mathbf{s}_k}$:

$$\boxed{H_{k+1} = V_k^T \, H_k \, V_k + \rho_k \, \mathbf{s}_k \mathbf{s}_k^T}$$

Properties:

1. Symmetric: $H_{k+1} = H_{k+1}^T$ (by construction)
2. Positive definite: if $H_k \succ 0$ and $\mathbf{y}_k^T \mathbf{s}_k > 0$
3. Inverse secant condition: $H_{k+1} \mathbf{y}_k = \mathbf{s}_k$ (verify by direct substitution)
4. Cost: $O(n^2)$ per update + $O(n^2)$ per direction computation

Verification of (3):

$$\begin{aligned} H_{k+1}\mathbf{y}_k &= (I - \rho_k \mathbf{s}_k \mathbf{y}_k^T) H_k (I - \rho_k \mathbf{y}_k \mathbf{s}_k^T)\mathbf{y}_k + \rho_k \mathbf{s}_k \mathbf{s}_k^T \mathbf{y}_k \\ &= (I - \rho_k \mathbf{s}_k \mathbf{y}_k^T) H_k \mathbf{y}_k \underbrace{(1 - \rho_k \mathbf{s}_k^T \mathbf{y}_k)}_{=0} + \rho_k \mathbf{s}_k \underbrace{(\mathbf{s}_k^T \mathbf{y}_k)}_{=1/\rho_k} = \mathbf{s}_k. \quad \checkmark \end{aligned}$$

Computing the search direction—just a matrix-vector product:

$$\mathbf{d}_{k+1} = -H_{k+1} \nabla f(\mathbf{x}_{k+1}) \qquad \text{cost: } O(n^2) \text{ vs. Newton's } O(n^3)$$

Worked Example: BFGS Inverse Update

Data: $H_1 = I_2, \quad \mathbf{s}_1 = (1,0)^T, \quad \mathbf{y}_1 = (2,1)^T, \quad \rho_1 = \frac{1}{2}.$

Step 1: Auxiliary matrices:

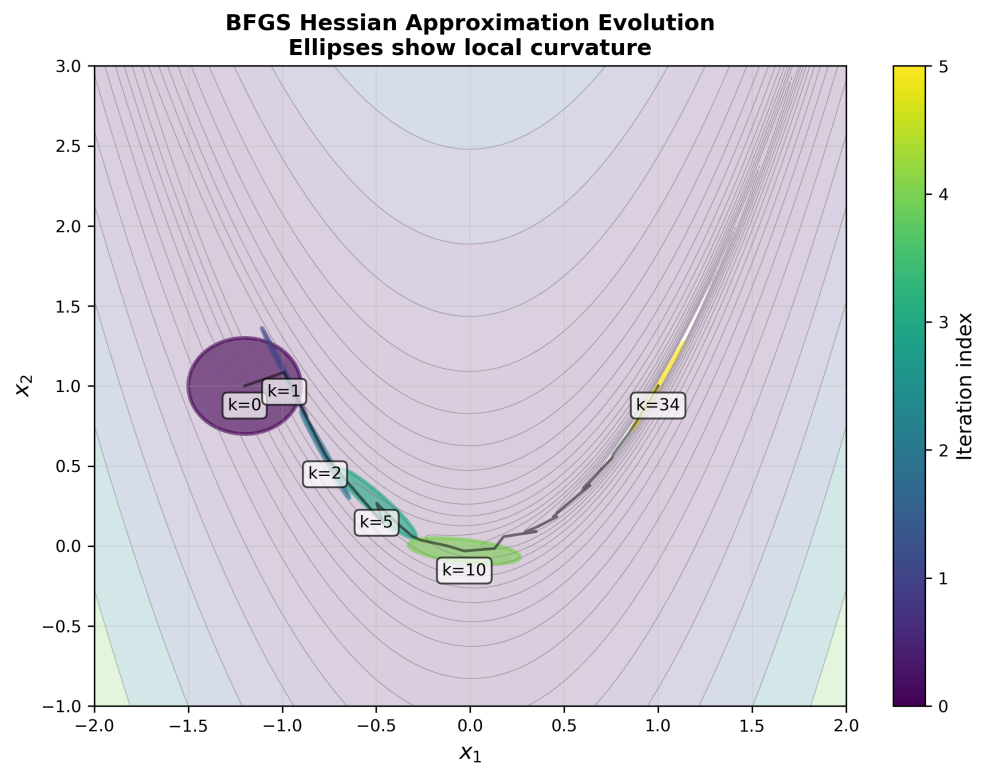
$$V_1 = I - \tfrac{1}{2}\mathbf{y}_1\mathbf{s}_1^T = \begin{pmatrix} 0 & 0 \\ -\frac{1}{2} & 1 \end{pmatrix}, \quad W_1 = I - \tfrac{1}{2}\mathbf{s}_1\mathbf{y}_1^T = \begin{pmatrix} 0 & -\frac{1}{2} \\ 0 & 1 \end{pmatrix}$$

Step 2: Compute H_2:

$$H_2 = W_1V_1 + \tfrac{1}{2}\mathbf{s}_1\mathbf{s}_1^T = \begin{pmatrix} \frac{1}{4} & -\frac{1}{2} \\ -\frac{1}{2} & 1 \end{pmatrix} + \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} \frac{3}{4} & -\frac{1}{2} \\ -\frac{1}{2} & 1 \end{pmatrix}$$

Step 3: Verify: $H_2\mathbf{y}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \mathbf{s}_1.$ ✓ Eigenvalues: $\frac{7 \pm \sqrt{17}}{8} > 0.$ ✓

How BFGS Learns the Curvature



Each ellipse shows $\{\mathbf{z} : \mathbf{z}^T B_k \mathbf{z} = 1\}$ at the corresponding iterate. Initially $B_0 = I$ (circle). Over iterations, the BFGS approximation deforms to match the true local curvature—elongating along steep directions and flattening along gentle ones.

Python: BFGS on the Rosenbrock Function

```python
import numpy as np
from scipy.optimize import line_search

f    = lambda x: (1-x[0])**2 + 100*(x[1]-x[0]**2)**2
grad = lambda x: np.array([-2*(1-x[0]) - 400*x[0]*(x[1]-x[0]**2),
                           200*(x[1]-x[0]**2)])

def bfgs(f, grad, x0, maxiter=300, tol=1e-8):
    n = len(x0);  x, g = x0.copy(), grad(x0)
    H = np.eye(n)                      # inverse Hessian approx
    for k in range(maxiter):
        d = -H @ g                     # search direction: O(n^2)
        res = line_search(f, grad, x, d, c1=1e-4, c2=0.9)
        alpha = res[0] if res[0] else 1e-4
        x_new = x + alpha * d
        g_new = grad(x_new)
        if np.linalg.norm(g_new) < tol: break

        s = x_new - x                  # step
        y = g_new - g                  # gradient change
        rho = 1.0 / (y @ s)            # > 0 by Wolfe conditions

        # BFGS inverse Hessian update
        I = np.eye(n)
        V = I - rho * np.outer(s, y)
        H = V @ H @ V.T + rho * np.outer(s, s)
        x, g = x_new, g_new
    return x, k+1

x0 = np.array([-1.2, 1.0])
x_opt, iters = bfgs(f, grad, x0)
print(f"BFGS: {iters} iters, f(x*) = {f(x_opt):.2e}")

# Or one line with scipy:
from scipy.optimize import minimize
res = minimize(f, x0, jac=grad, method='BFGS')
```

The Complete BFGS Algorithm

Algorithm: BFGS with Wolfe Line Search
Input: Starting point $\mathbf{x}_1$, tolerance $\epsilon > 0$. Set $H_1 = I_n$, $k = 1$.

1. If $\|\nabla f(\mathbf{x}_k)\| < \epsilon$: **stop**.
2. $\mathbf{d}_k = -H_k \nabla f(\mathbf{x}_k)$ [$O(n^2)$ mat-vec]
3. Find λ_k satisfying Wolfe conditions [backtracking]
4. $\mathbf{x}_{k+1} = \mathbf{x}_k + \lambda_k \mathbf{d}_k$
5. $\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k, \quad \mathbf{y}_k = \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k)$
6. $\rho_k = \frac{1}{\mathbf{y}_k^T \mathbf{s}_k}$
7. $H_{k+1} = (I - \rho_k \mathbf{s}_k \mathbf{y}_k^T) H_k (I - \rho_k \mathbf{y}_k \mathbf{s}_k^T) + \rho_k \mathbf{s}_k \mathbf{s}_k^T$ [$O(n^2)$]
8. $k \leftarrow k + 1$, go to Step 1.

Key: The Wolfe line search guarantees $\mathbf{y}_k^T \mathbf{s}_k > 0$, maintaining positive definiteness and ensuring global convergence.

Worked Example: BFGS on $f(\mathbf{x}) = x_1^2 + 4x_2^2$

Minimize $f(\mathbf{x}) = x_1^2 + 4x_2^2$ from $\mathbf{x}_0 = (2, 1)^T$. True minimum: $\mathbf{x}^* = (0, 0)^T$.

Gradient: $\nabla f(\mathbf{x}) = (2x_1,\, 8x_2)^T$. Hessian: $\nabla^2 f = \mathrm{diag}(2, 8)$. $\kappa = 4$.

Iteration 0: $H_0 = I$, $\mathbf{g}_0 = (4, 8)^T$, $\mathbf{d}_0 = -H_0\mathbf{g}_0 = (-4, -8)^T$.

Line search:
$\alpha_0 = \arg\min_\alpha f(\mathbf{x}_0 + \alpha\mathbf{d}_0) = \arg\min_\alpha[(2 - 4\alpha)^2 + 4(1 - 8\alpha)^2]$.
Setting derivative to 0: $\alpha_0 = \frac{9}{136} \approx 0.0662$.

$\mathbf{x}_1 = (2 - 4 \cdot \frac{9}{136},\, 1 - 8 \cdot \frac{9}{136})^T = (\frac{236}{136},\, \frac{64}{136})^T \approx (1.735,\, 0.471)^T$

Update $H_0 \to H_1$: $\mathbf{s}_0 = \mathbf{x}_1 - \mathbf{x}_0 \approx (-0.265, -0.529)^T$,
$\mathbf{y}_0 = \nabla f(\mathbf{x}_1) - \mathbf{g}_0 \approx (-0.529, -4.235)^T$

$\rho_0 = \frac{1}{\mathbf{y}_0^T\mathbf{s}_0} \approx \frac{1}{2.382}$. Apply the BFGS formula:

$$H_1 = (I - \rho_0\mathbf{s}_0\mathbf{y}_0^T)H_0(I - \rho_0\mathbf{y}_0\mathbf{s}_0^T) + \rho_0\mathbf{s}_0\mathbf{s}_0^T$$

$H_1 \approx \begin{pmatrix} 0.426 & -0.051 \\ -0.051 & 0.097 \end{pmatrix}$. Compare with true $(\nabla^2 f)^{-1} = \mathrm{diag}(0.5, 0.125)$—already a reasonable approximation after one step!

Iteration 1: $\mathbf{d}_1 = -H_1\mathbf{g}_1$. On a 2D quadratic, BFGS with exact line search converges in at most 2 steps (like CG). After this step: $\mathbf{x}_2 \approx \mathbf{x}^*$ and $H_2 \approx (\nabla^2 f)^{-1}$.

Convergence of BFGS: Global and Superlinear

BFGS enjoys the strongest convergence guarantees of any quasi-Newton method:

Theorem (Thm 6.5: Global convergence of BFGS (Powell))
Let f be twice continuously differentiable and strongly convex on the level set $\mathcal{L} = \{\mathbf{x} : f(\mathbf{x}) \leq f(\mathbf{x}_0)\}$, with $m\|\mathbf{z}\|^2 \leq \mathbf{z}^T \nabla^2 f(\mathbf{x}) \mathbf{z} \leq M\|\mathbf{z}\|^2$ for all $\mathbf{x} \in \mathcal{L}$. Then the BFGS method with Wolfe line search converges: $\mathbf{x}_k \to \mathbf{x}^*$.

Proof idea: Analyze the function $\psi(B) = \operatorname{trace}(B) - \ln \det(B)$, which measures how far B_k is from the identity. Show that $\psi(B_{k+1})$ grows at most linearly, but $\sum \cos^2 \theta_k$ must diverge, forcing $\cos \theta_k \to 0$ cannot hold—contradiction.

Theorem (Thm 6.6: Superlinear convergence of BFGS)
Under the additional assumption that $\nabla^2 f$ is Lipschitz continuous at $\mathbf{x}^*$, and provided $\sum_{k=1}^{\infty} \|\mathbf{x}_k - \mathbf{x}^*\| < \infty$, the BFGS iterates converge superlinearly:

$$\lim_{k \to \infty} \frac{\|\mathbf{x}_{k+1} - \mathbf{x}^*\|}{\|\mathbf{x}_k - \mathbf{x}^*\|} = 0$$

Superlinear convergence means the rate of convergence improves with each iteration—the method accelerates as it approaches the solution.

The Dennis–Moré Characterization (Thm 6.6 Proof Sketch)

Key question: What exactly makes a quasi-Newton method superlinearly convergent?

Theorem (Dennis–Moré Characterization (Thm 3.6 / 6.6))
A quasi-Newton method $\mathbf{x}_{k+1} = \mathbf{x}_k - B_k^{-1}\nabla f(\mathbf{x}_k)$ *converges superlinearly if and only if:*

$$\lim_{k\to\infty} \frac{\|(B_k - \nabla^2 f(\mathbf{x}^*))\mathbf{d}_k\|}{\|\mathbf{d}_k\|} = 0$$

where $\mathbf{d}_k = \mathbf{x}_{k+1} - \mathbf{x}_k$.

What this means: B_k does not need to converge to $\nabla^2 f(\mathbf{x}^*)$ in norm. It only needs to act like the true Hessian along the search direction $\mathbf{d}_k$. This is a much weaker condition!

Why BFGS satisfies this: The proof uses a potential function argument.

Step 1: Define scaled quantities $\tilde{\mathbf{s}}_k = G_*^{1/2}\mathbf{s}_k$, $\tilde{\mathbf{y}}_k = G_*^{-1/2}\mathbf{y}_k$, $\tilde{B}_k = G_*^{-1/2}B_k G_*^{-1/2}$ where $G_* = \nabla^2 f(\mathbf{x}^*)$.

Step 2: The potential $\psi(\tilde{B}_k) = \operatorname{tr}(\tilde{B}_k) - \ln\det(\tilde{B}_k)$ satisfies $\psi(\tilde{B}_k) \geq n$ with equality iff $\tilde{B}_k = I$ (i.e., $B_k = G_*$).

Step 3: Show $0 < \psi(\tilde{B}_{k+1}) \leq \psi(\tilde{B}_k) + 3c\epsilon_k + \ln\cos^2\tilde{\theta}_k + [1 - \tilde{q}_k/\cos^2\tilde{\theta}_k + \ln(\tilde{q}_k/\cos^2\tilde{\theta}_k))]$ where the bracketed term is ≤ 0 (since $1 - t + \ln t \leq 0$ for all $t > 0$).

Step 4: Sum over k. Since $\sum \epsilon_k < \infty$ (by eq. 6.52), the sum $\sum \ln\cos^2\tilde{\theta}_k$ must converge, forcing $\cos\tilde{\theta}_k \to 1$ and $\tilde{q}_k \to 1$. These imply the Dennis–Moré condition. □

The Self-Correcting Property of BFGS

A remarkable feature distinguishes BFGS from other quasi-Newton methods:

Self-Correcting Property
If the BFGS matrix H_k becomes a poor approximation to the inverse Hessian—even very poor—the algorithm tends to correct itself within a few steps.

Why this happens: When H_k is inaccurate, the resulting step slows down the iteration. But the Wolfe line search samples curvature at points that expose the mismatch, and the rank-2 update incorporates this new curvature information. The "forget + learn" structure of the BFGS update naturally washes out old errors.

Contrast with DFP: The DFP formula does *not* have this self-correcting property as strongly. Bad Hessian approximations tend to persist, which is one reason BFGS has largely replaced DFP in practice.

Caveat: Self-correction requires the Wolfe conditions. With a simple backtracking line search (Armijo only), the curvature condition $\mathbf{y}_k^T \mathbf{s}_k > 0$ may fail, degrading BFGS performance.

Practical BFGS: The Initial Scaling Heuristic

Problem: Starting with $H_0 = I$ can be a poor initial guess if the true Hessian is far from the identity.

Solution: After the first step, rescale H_0 before applying the first update:

$$H_0 \leftarrow \frac{\mathbf{y}_0^T \mathbf{s}_0}{\mathbf{y}_0^T \mathbf{y}_0} I$$

Why this works: The scalar $\mathbf{y}_0^T \mathbf{s}_0 / \mathbf{y}_0^T \mathbf{y}_0$ approximates the reciprocal of an eigenvalue of the average Hessian $\bar{G}_0$ along the first step. This makes H_0 the right order of magnitude, so the first BFGS update H_1 starts from a reasonable approximation.

Impact: Without this scaling, the first few steps can be wildly off-scale, requiring many function evaluations to find acceptable step lengths. With it, $\alpha_k = 1$ is typically accepted immediately in the line search.

This same scaling idea is used in L-BFGS as the initial matrix $H_k^0 = \gamma_k I$ at each iteration, where $\gamma_k = \mathbf{s}_{k-1}^T \mathbf{y}_{k-1} / \mathbf{y}_{k-1}^T \mathbf{y}_{k-1}$.

The DFP Update and the Broyden Family

BFGS is not the only quasi-Newton update. The **DFP formula** (Davidon–Fletcher–Powell, 1959) is its "dual," obtained by swapping $\mathbf{s}_k \leftrightarrow \mathbf{y}_k$ and $B_k \leftrightarrow H_k$:

$$H_{k+1}^{\text{DFP}} = H_k - \frac{H_k \mathbf{y}_k \mathbf{y}_k^T H_k}{\mathbf{y}_k^T H_k \mathbf{y}_k} + \frac{\mathbf{s}_k \mathbf{s}_k^T}{\mathbf{y}_k^T \mathbf{s}_k}$$

Both DFP and BFGS belong to the Broyden family, parameterized by $\phi_k \in [0, 1]$:

$$B_{k+1} = (1 - \phi_k) B_{k+1}^{\text{BFGS}} + \phi_k B_{k+1}^{\text{DFP}}$$

Theorem (Thm 6.4: Broyden class on quadratics)
On a strongly convex quadratic with exact line search, all members of the restricted Broyden class ($\phi_k \in [0, 1]$):

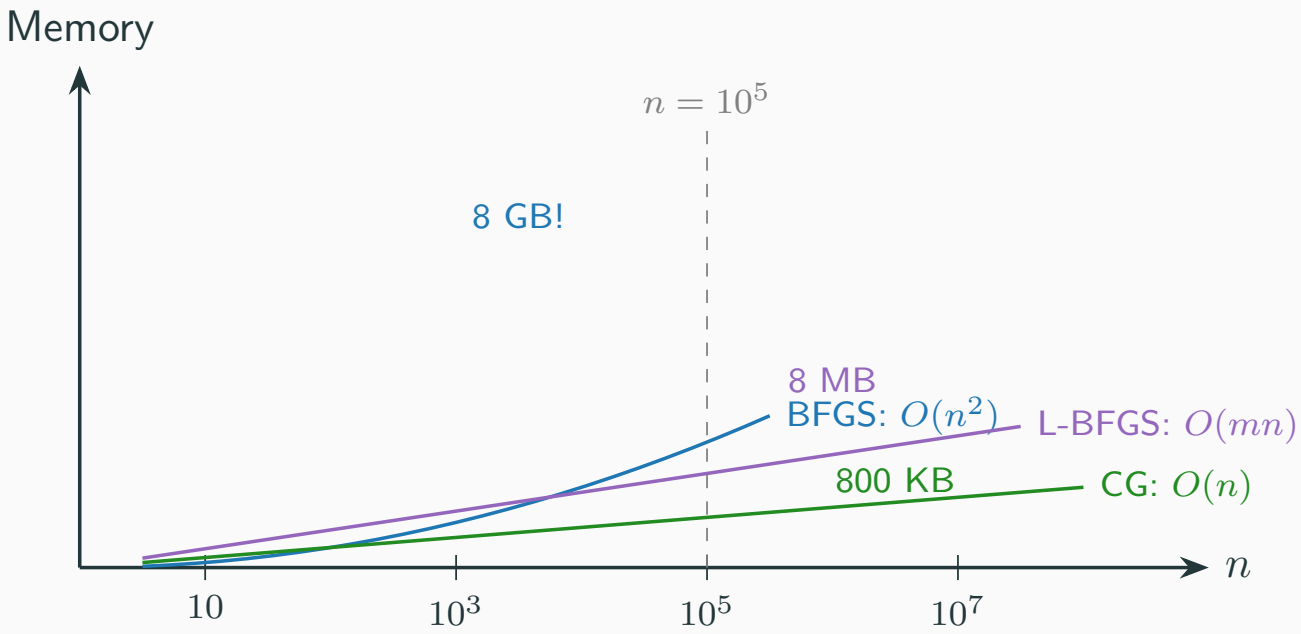
1. *Generate the same iterates (independent of ϕ_k!)*
2. *Satisfy the secant equation along all previous directions*
3. *With $B_0 = I$: produce iterates identical to CG*
4. *Converge in at most n steps with $B_n = A$ (the true Hessian)*

So why prefer BFGS? On nonquadratic problems with inexact line search, BFGS has the strongest self-correcting properties and the best empirical performance.

Roadmap

The Memory Problem

Issue: BFGS stores $H_k \in \mathbb{R}^{n \times n}$, requiring $O(n^2)$ memory.



L-BFGS idea: Don't store H_k explicitly. Instead, store only the last m pairs $\{(\mathbf{s}_i, \mathbf{y}_i)\}_{i=k-m}^{k-1}$ and implicitly compute $H_k \nabla f(\mathbf{x}_k)$.

Typical m: 3–20 (usually $m = 10$ works well).

The Two-Loop Recursion

Key insight: The BFGS inverse update is a recursive formula. We can "unroll" m levels:

$$H_k \mathbf{g}_k = V_{k-1}^T \cdots V_{k-m}^T \, H_k^0 \, V_{k-m} \cdots V_{k-1} \mathbf{g}_k + \text{rank-1 corrections}$$

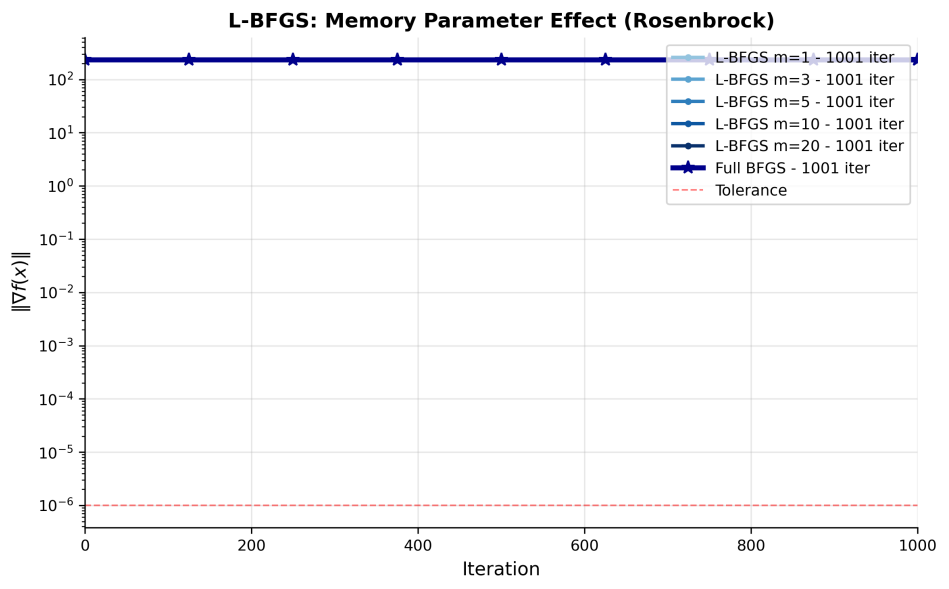
where $H_k^0 = \frac{\mathbf{s}_{k-1}^T \mathbf{y}_{k-1}}{\mathbf{y}_{k-1}^T \mathbf{y}_{k-1}} I$ is a simple scaling.

L-BFGS Two-Loop Recursion (Nocedal, 1980)

1. $\mathbf{z} \leftarrow \mathbf{g}_k$
2. Loop 1 (backward): For $i = k-1, \ldots, k-m$:
   $\alpha_i = \rho_i \mathbf{s}_i^T \mathbf{z}; \quad \mathbf{z} \leftarrow \mathbf{z} - \alpha_i \mathbf{y}_i$
3. Scale: $\mathbf{h} \leftarrow H_k^0 \mathbf{z}$
4. Loop 2 (forward): For $i = k-m, \ldots, k-1$:
   $\beta = \rho_i \mathbf{y}_i^T \mathbf{h}; \quad \mathbf{h} \leftarrow \mathbf{h} + (\alpha_i - \beta)\mathbf{s}_i$
5. Return $\mathbf{d}_k = -\mathbf{h}$

Cost: $O(mn)$ **Memory:** $O(mn)$

L-BFGS: Effect of Memory Parameter m



Observations:

- $m = 1$: barely better than steepest descent
- $m = 5$–10: substantial improvement, approaching full BFGS
- $m = 20$: nearly identical to full BFGS on this 2D problem
- Diminishing returns beyond $m \approx 10$–20 in practice

Python: L-BFGS Two-Loop Recursion

```python
def lbfgs(f, grad, x0, m=10, maxiter=300, tol=1e-8):
    """Algorithm 7.4 from Nocedal & Wright."""
    x, g = x0.copy(), grad(x0)
    S, Y, rhos = [], [], []        # last m pairs {s_i, y_i}

    for k in range(maxiter):
        # --- Two-loop recursion: compute H_k @ g ---
        q = g.copy();  alphas = []
        for si, yi, ri in zip(reversed(S), reversed(Y), reversed(rhos)):
            a = ri * (si @ q);  q -= a * yi;  alphas.append(a)
        alphas.reverse()
        # Initial scaling H0 = gamma * I
        gamma = (S[-1]@Y[-1])/(Y[-1]@Y[-1]) if S else 1.0
        r = gamma * q
        for i, (si, yi, ri) in enumerate(zip(S, Y, rhos)):
            b = ri * (yi @ r);  r += (alphas[i] - b) * si
        d = -r
        # --- Line search + update ---
        from scipy.optimize import line_search
        res = line_search(f, grad, x, d, c1=1e-4, c2=0.9)
        alpha = res[0] if res[0] else 1e-4
        x_new, g_new = x + alpha*d, grad(x + alpha*d)
        if np.linalg.norm(g_new) < tol: break
        s, y = x_new - x, g_new - g
        S.append(s); Y.append(y); rhos.append(1.0/(y@s))
        if len(S) > m: S.pop(0); Y.pop(0); rhos.pop(0)
        x, g = x_new, g_new
    return x, k+1

# scipy one-liner (L-BFGS-B is the default for large problems):
from scipy.optimize import minimize
res = minimize(f, x0, jac=grad, method='L-BFGS-B',
               options={'maxcor': 10})  # maxcor = m
```

Memoryless BFGS = Hestenes–Stiefel CG (§7.2)

A beautiful connection: L-BFGS with $m = 1$ and $H_k^0 = I$ is exactly a conjugate gradient method.

Step 1: Memoryless BFGS. With $m = 1$, L-BFGS keeps only the most recent pair $(\mathbf{s}_k, \mathbf{y}_k)$ and resets to $H_k^0 = I$ each iteration. The search direction is:

$$\mathbf{d}_{k+1} = -H_{k+1}\nabla f_{k+1} \quad \text{where} \quad H_{k+1} = \left(I - \frac{\mathbf{s}_k\mathbf{y}_k^T}{\mathbf{y}_k^T\mathbf{s}_k}\right)\left(I - \frac{\mathbf{y}_k\mathbf{s}_k^T}{\mathbf{y}_k^T\mathbf{s}_k}\right) + \frac{\mathbf{s}_k\mathbf{s}_k^T}{\mathbf{y}_k^T\mathbf{s}_k}$$

(one BFGS update applied to I).

Step 2: Expand the product. Multiply out $\mathbf{d}_{k+1} = -H_{k+1}\nabla f_{k+1}$:

$$\mathbf{d}_{k+1} = -\nabla f_{k+1} + \frac{\nabla f_{k+1}^T\mathbf{y}_k}{\mathbf{y}_k^T\mathbf{s}_k}\,\mathbf{s}_k + \frac{\nabla f_{k+1}^T\mathbf{s}_k}{\mathbf{y}_k^T\mathbf{s}_k}\,\mathbf{y}_k - \frac{\nabla f_{k+1}^T\mathbf{s}_k \cdot \mathbf{y}_k^T\mathbf{y}_k}{(\mathbf{y}_k^T\mathbf{s}_k)^2}\,\mathbf{s}_k + \frac{\nabla f_{k+1}^T\mathbf{s}_k}{\mathbf{y}_k^T\mathbf{s}_k}\,\mathbf{s}_k$$

Step 3: Assume exact line search. Then $\nabla f_{k+1}^T\mathbf{d}_k = 0$, so $\nabla f_{k+1}^T\mathbf{s}_k = \alpha_k\nabla f_{k+1}^T\mathbf{d}_k = 0$. All terms with $\nabla f_{k+1}^T\mathbf{s}_k$ vanish, leaving:

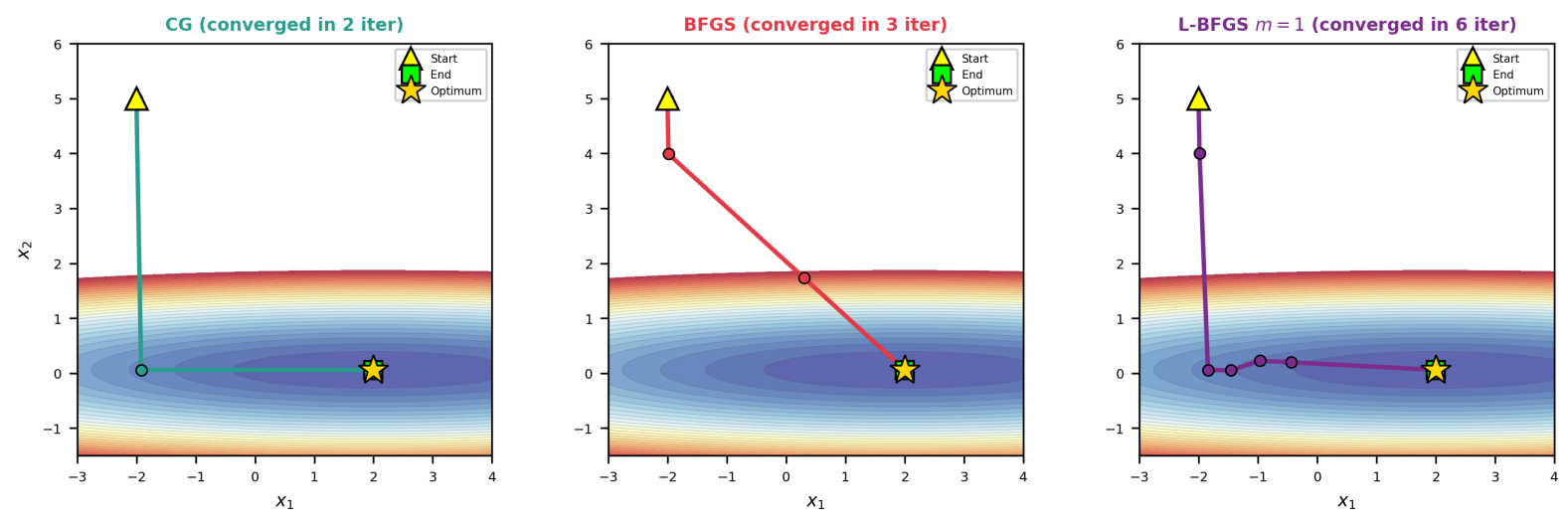
$$\boxed{\mathbf{d}_{k+1} = -\nabla f_{k+1} + \frac{\nabla f_{k+1}^T\mathbf{y}_k}{\mathbf{y}_k^T\mathbf{d}_k}\,\mathbf{d}_k}$$

This is the Hestenes–Stiefel conjugate gradient formula with $\beta_{k+1}^{HS} = \frac{\nabla f_{k+1}^T\mathbf{y}_k}{\mathbf{y}_k^T\mathbf{d}_k}$ (eq. 7.23).

L-BFGS and nonlinear CG are not separate families—CG is the $m = 1$ limit of L-BFGS. As m increases from 1 to n, we interpolate continuously from CG to full BFGS.

Roadmap

On an Ill-Conditioned Quadratic



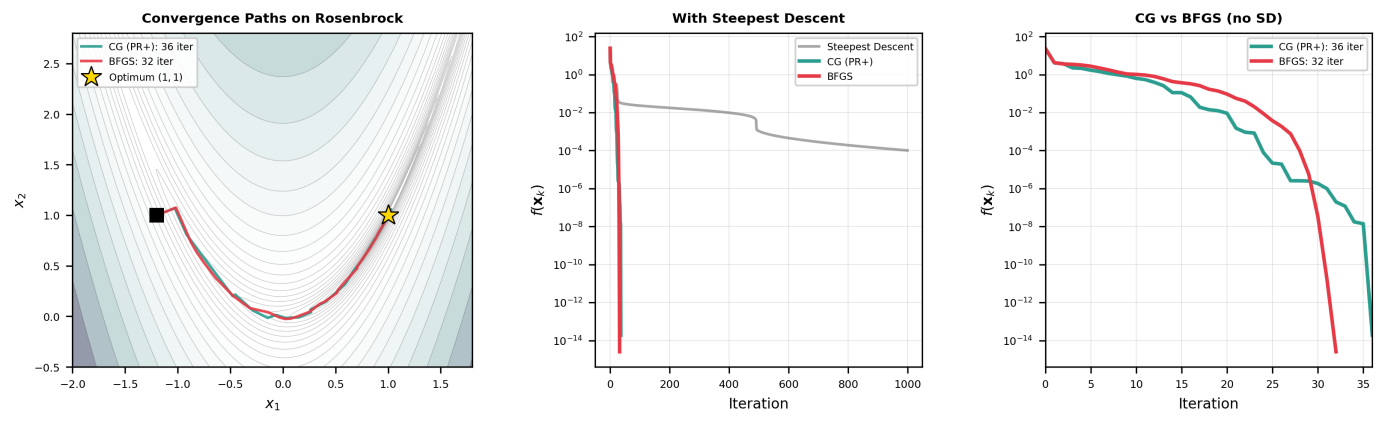
$f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A\mathbf{x} - \mathbf{b}^T\mathbf{x}$ with $A = \mathrm{diag}(1, 50)$, $\kappa = 50$.

CG: Converges in exactly 2 steps (finite termination on quadratics).

BFGS: Takes a few more iterations but builds curvature approximation.

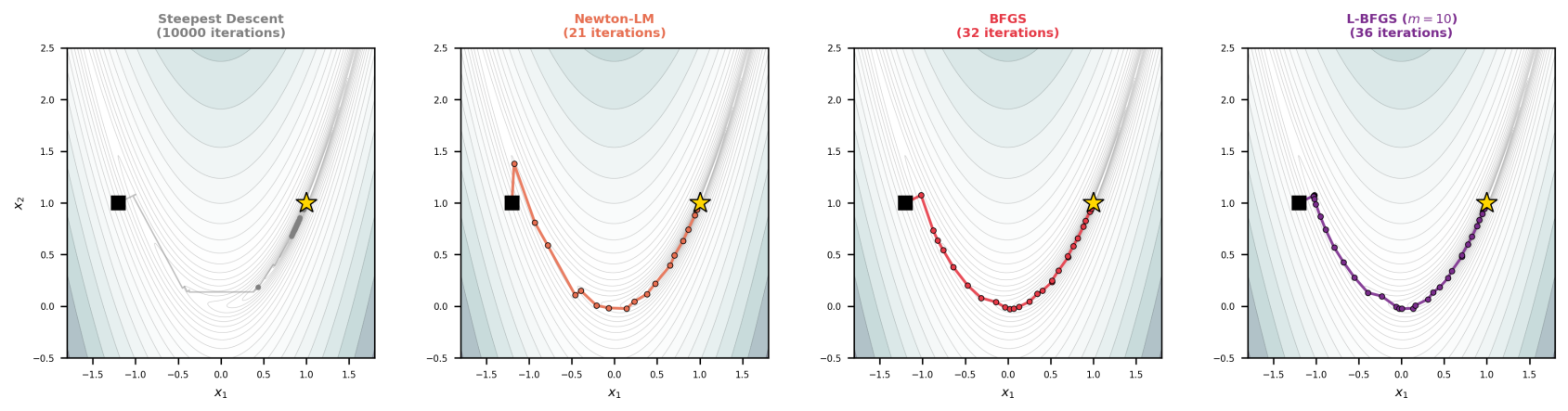
L-BFGS ($m = 1$): Limited memory hampers convergence—path is less direct.

On the Rosenbrock Function



Key difference: BFGS accumulates a full curvature model, which helps it navigate the curved valley more efficiently. Nonlinear CG must rely on 1D line searches and periodic restarts.

All Methods on the Rosenbrock Function



$f(\mathbf{x}) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$ from $\mathbf{x}_0 = (-1.2, 1)^T$. Steepest descent zigzags badly; Newton (with LM) converges rapidly; BFGS is nearly as fast; L-BFGS trades a little speed for much less memory.

Python: Head-to-Head Comparison with SciPy

```python
import numpy as np
from scipy.optimize import minimize

f    = lambda x: (1-x[0])**2 + 100*(x[1]-x[0]**2)**2
grad = lambda x: np.array([-2*(1-x[0]) - 400*x[0]*(x[1]-x[0]**2),
                           200*(x[1]-x[0]**2)])
hess = lambda x: np.array([[2-400*(x[1]-3*x[0]**2), -400*x[0]],
                           [-400*x[0],                200    ]])
x0 = np.array([-1.2, 1.0])

results = {
    'CG':         minimize(f, x0, jac=grad, method='CG'),
    'BFGS':       minimize(f, x0, jac=grad, method='BFGS'),
    'L-BFGS':     minimize(f, x0, jac=grad, method='L-BFGS-B',
                           options={'maxcor': 10}),
    'Newton-CG': minimize(f, x0, jac=grad, hess=hess,
                           method='Newton-CG'),
}

for name, res in results.items():
    print(f"{name:10s}: {res.nit:3d} iters, "
          f"{res.nfev:3d} f-evals, f* = {res.fun:.2e}")
# CG         :  37 iters,  94 f-evals, f* = 1.37e-15
# BFGS       :  34 iters,  40 f-evals, f* = 4.02e-18
# L-BFGS     :  25 iters,  30 f-evals, f* = 2.44e-16
# Newton-CG :  32 iters,  36 f-evals, f* = 1.26e-14
```

Try it: Use `scipy.optimize.rosen` for n-dimensional Rosenbrock!

What Is SciPy Actually Doing?

When you call `scipy.optimize.minimize`, what runs under the hood?

Method	Implementation
`'CG'`	Pure Python/NumPy. Polak–Ribière with automatic restarts. Line search via `scalar_search_wolfe2` (strong Wolfe conditions).
`'BFGS'`	Pure Python/NumPy. Classic BFGS with inverse Hessian update. Same Wolfe line search. Returns approximate inverse Hessian in `res.hess_inv`.
`'L-BFGS-B'`	Compiled C (translated from Fortran). The original Zhu, Byrd, Lu & Nocedal code (Algorithm 778, *ACM TOMS*, 1997). Supports bound constraints. This is the fastest option in SciPy.
`'Newton-CG'`	Pure Python. Truncated Newton: uses CG as the inner solver for the Newton system $H_k \mathbf{d} = -\mathbf{g}_k$.

L-BFGS-B is the only method backed by compiled code—it is significantly faster per iteration than `'CG'` or `'BFGS'` for large n.

Is SciPy the Best Choice?

When SciPy is great:

- Small to medium-scale problems ($n \lesssim 10^4$)
- Rapid prototyping and research
- The L-BFGS-B implementation is mature, well-tested, and hard to beat on CPU

When you need something else:

Scenario	Better tool
GPU/TPU acceleration	JAX (`jaxopt.LBFGS`): JIT-compiled, differentiable, runs on GPU/TPU natively
Deep learning pipeline	PyTorch (`torch.optim.LBFGS`): integrates with autograd; useful for full-batch training
Real-time / embedded	C++ or Rust: sub-millisecond solve times; `OpEn` (Rust) achieves 10–100$\times$ speedup over interior point
Julia ecosystem	Optim.jl: multiple algorithms, type-stable, competitive with SciPy

For this course, SciPy is the right tool. For production ML at scale, consider JAX or PyTorch.

What About CVXPY?

Short answer: CVXPY is a different paradigm from `scipy.optimize`.

	scipy.optimize	**CVXPY**
Problem type	General nonlinear $\min f(\mathbf{x})$	Disciplined convex programs
Methods	CG, BFGS, L-BFGS, Newton	Interior point, ADMM
Solvers	Built-in	ECOS, SCS, MOSEK, Gurobi
Input	$f(\mathbf{x})$ and $\nabla f(\mathbf{x})$	Symbolic problem description
Reformulation	None	Converts to conic form
Constraints	Limited (bounds, eq.)	LP, QP, SOCP, SDP

CVXPY does not provide CG, BFGS, or L-BFGS. It reformulates your problem into standard conic form and dispatches to specialized solvers.

Use CVXPY when your problem is a structured convex program (LP, QP, SDP). Use `scipy.optimize` when you have a general objective with a gradient.

Should You Write Your Own?

When to use an existing library (almost always):

- Line searches are tricky to get right (Wolfe conditions, safeguards)
- Decades of numerical debugging in SciPy/L-BFGS-B
- Your time is better spent on modeling, not reimplementing BFGS

When writing your own makes sense:

- Education (this course!): implementing CG and BFGS builds deep understanding
- Problem-specific structure: your Hessian-vector product has special sparsity or you can exploit parallelism that generic solvers miss
- Embedded / real-time: C++ or Rust for $< 1\,\text{ms}$ solve times on microcontrollers
- Research: testing new line search rules, new β_k formulas, or hybrid methods

For homework: implement from scratch to learn. For research: start with SciPy, then optimize the bottleneck. For production: use the compiled, tested, community-maintained code.

Numerical Stability: CG in Finite Precision

In exact arithmetic, CG converges in $\leq n$ steps with perfectly orthogonal residuals and A-conjugate directions. In floating point, things go wrong:

Loss of orthogonality. After many iterations, $\mathbf{g}_i^T \mathbf{g}_j \neq 0$ due to rounding. The directions lose A-conjugacy, and CG may "stall"—the iterates stop improving even though $\|\mathbf{g}_k\|$ is not small.

Practical consequences:

- On a problem with $n = 1000$, CG may need $\gg n$ iterations due to loss of conjugacy
- The convergence bound $\|\mathbf{x}_k - \mathbf{x}^*\|_A \leq 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k \|\mathbf{x}_0 - \mathbf{x}^*\|_A$ still approximately holds, but the finite termination property ($\mathbf{x}_n = \mathbf{x}^*$) is lost
- In practice, CG still converges—it just may take more than n iterations

Remedies:

- Preconditioning is the primary fix: if $\kappa(M^{-1}A)$ is small, CG converges long before rounding becomes an issue
- Reorthogonalization (Lanczos-style): periodically re-orthogonalize residuals. Expensive ($O(kn)$ per step) but sometimes necessary for eigenvalue computations
- For the iterative solve itself: usually just run more iterations—the convergence bound doesn't care about orthogonality loss

Numerical Stability: Nonlinear CG Restarts

For nonlinear CG, the situation is different. Since the function is not quadratic, "conjugacy" is only approximate even in exact arithmetic. Practical strategies:

Restart strategies:

- Periodic restart (Powell, 1977): Reset $\mathbf{d}_k = -\mathbf{g}_k$ every n iterations. Simple but wasteful—may discard useful information.
- Gradient-based restart: Reset when $|\mathbf{g}_{k+1}^T \mathbf{g}_k| > \nu \|\mathbf{g}_{k+1}\|^2$ for some $\nu \in (0.1, 0.2)$. This detects when gradients are no longer approximately orthogonal (loss of conjugacy).
- PR+ automatic restart: Polak–Ribière+ with $\beta_k^+ = \max(\beta_k^{PR}, 0)$ effectively restarts whenever $\beta_k^{PR} < 0$. When gradients become nearly parallel (flat region), $\mathbf{g}_{k+1}^T \mathbf{y}_k \approx 0$, so $\beta \approx 0$ and the method automatically restarts toward steepest descent.

Line search sensitivity:

- Nonlinear CG is much more sensitive to line search quality than BFGS
- Use the strong Wolfe conditions with $c_2 = 0.1$ (not 0.9!)
- With a sloppy line search, FR can generate directions nearly orthogonal to $-\mathbf{g}_k$ (very short steps), while PR+ self-corrects

In practice, use PR+ with strong Wolfe ($c_2 = 0.1$) and a gradient-based restart check. This combination is robust and rarely needs manual tuning.

Numerical Stability: BFGS Implementation Tips

BFGS is remarkably robust, but there are implementation details that matter:

1. The Shanno–Phua scaling heuristic. After the first step, rescale H_0 before applying the first BFGS update:

$$H_0 \leftarrow \frac{\mathbf{y}_0^T \mathbf{s}_0}{\mathbf{y}_0^T \mathbf{y}_0} I$$

Without this, the first search direction can be wildly off-scale, causing the line search to reject $\alpha = 1$ and wasting function evaluations.

2. Guarding against $\mathbf{y}_k^T \mathbf{s}_k \approx 0$. Even with Wolfe line search, floating-point arithmetic can produce very small $\mathbf{y}_k^T \mathbf{s}_k$. If $\rho_k = 1/(\mathbf{y}_k^T \mathbf{s}_k)$ is too large, skip the update: keep $H_{k+1} = H_k$ and proceed. L-BFGS-B in SciPy does this.

3. Damped BFGS. For nonconvex problems where $\mathbf{y}_k^T \mathbf{s}_k > 0$ may not hold (even with Wolfe), Powell's damped BFGS modifies $\mathbf{y}_k$:

$$\hat{\mathbf{y}}_k = \theta_k \mathbf{y}_k + (1-\theta_k) B_k \mathbf{s}_k, \qquad \theta_k = \begin{cases} 1 & \text{if } \mathbf{y}_k^T \mathbf{s}_k \geq 0.2\, \mathbf{s}_k^T B_k \mathbf{s}_k \\ \frac{0.8\, \mathbf{s}_k^T B_k \mathbf{s}_k}{\mathbf{s}_k^T B_k \mathbf{s}_k - \mathbf{y}_k^T \mathbf{s}_k} & \text{otherwise} \end{cases}$$

This interpolates between the true $\mathbf{y}_k$ and the model prediction $B_k \mathbf{s}_k$ to guarantee $\hat{\mathbf{y}}_k^T \mathbf{s}_k > 0$.

4. L-BFGS storage. Store vectors $\mathbf{s}_i, \mathbf{y}_i$ and scalars ρ_i in circular buffers. Never form H_k explicitly—use the two-loop recursion. Total storage: $2mn + O(m)$ scalars.

Practical Summary: Which Method, Which Settings?

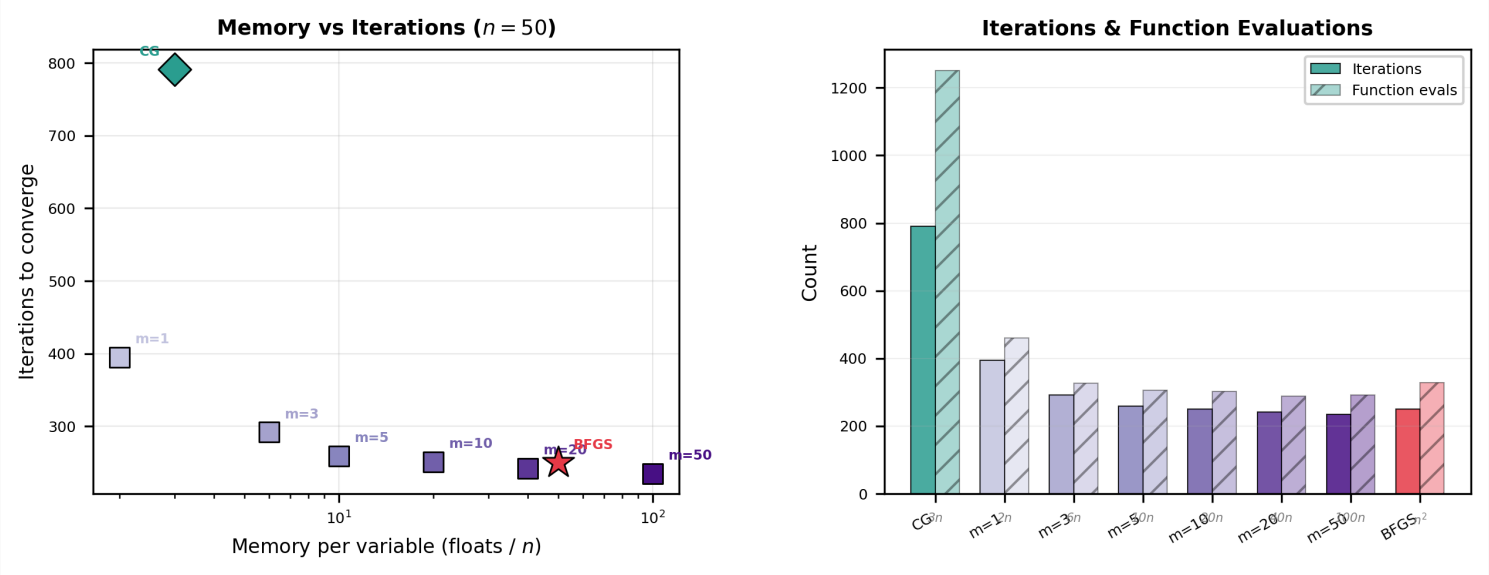
Method	Storage	Rate	Line search	When to use
SD	$O(n)$	linear	any	Never (for comparison only)
CG (linear)	$O(n)$	finite	exact	Sparse $A\mathbf{x} = \mathbf{b}$, $n \gg 10^4$
CG (nonlinear)	$O(n)$	linear	strong Wolfe	n too large for $O(n^2)$
BFGS	$O(n^2)$	superlinear	Wolfe	$n \lesssim 1000$, smooth f
L-BFGS	$O(mn)$	superlinear	Wolfe	$n \gg 1000$, smooth f
Newton-CG	$O(n)$	quadratic	Wolfe	Have $\nabla^2 f$ or Hess-vec

Default choices:

- Small–medium problems ($n \leq 1000$): BFGS with Wolfe ($c_1 = 10^{-4}$, $c_2 = 0.9$), $H_0 = I$ with Shanno–Phua scaling. This is `scipy.optimize.minimize(method='BFGS')`.
- Large problems ($n > 1000$): L-BFGS with $m = 10$, same Wolfe parameters. This is `scipy.optimize.minimize(method='L-BFGS-B')`.
- Very large sparse ($n > 10^5$, sparse Hessian): Newton-CG or truncated Newton with preconditioned CG as the inner solver.
- Noisy gradients (stochastic): Use SGD, Adam, or L-BFGS with variance reduction—standard BFGS fails with noisy gradients.

When in doubt, try L-BFGS first. It is the most broadly applicable method with the best performance-to-effort ratio.

The Memory–Convergence Tradeoff



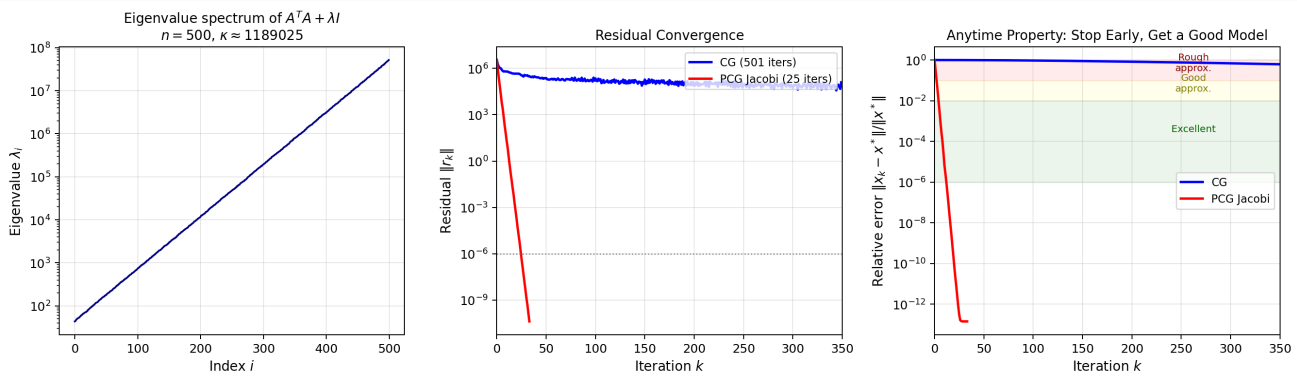
$n = 50$ Rosenbrock function. Left: memory per variable vs. iterations—CG uses only $3n$ storage but needs ~ 800 iterations; BFGS uses n^2 but converges in ~ 250. L-BFGS with $m = 10$–20 is the sweet spot: nearly matching BFGS speed at a fraction of the memory. Right: bar chart showing both iterations and function evaluations.

Roadmap

Case Study 1: Ridge Regression (CG for Linear Systems)

Problem: Predict housing prices from $p = 500$ features measured on $n = 5{,}000$ houses. Features have wildly different scales (square footage vs. number of bedrooms vs. zip code).

The ridge regression estimate solves $(A^TA + \lambda I)\mathbf{x} = A^T\mathbf{b}$, a 500×500 SPD system. Due to the scale mismatch, $\kappa \approx 10^6$.



Left: Eigenvalue spectrum spans 6 orders of magnitude. **Center:** Plain CG stalls (> 500 iters); Jacobi preconditioning (just rescale each feature by $1/\sqrt{a_{ii}}$) converges in 25 iterations. **Right:** After only 4 PCG iterations, the solution is within 1% of optimal—you can stop early and still get a good model.

Case Study 1: What We Learn

Why is Jacobi preconditioning so effective here?

The huge condition number ($\kappa \approx 10^6$) comes entirely from the column scaling: features measured in different units create a diagonal dominance mismatch. Jacobi preconditioning ($M = \text{diag}(A^T A + \lambda I)$) removes this, reducing $\kappa(M^{-1}A)$ to something manageable.

In practice, this is just standardizing your features. When you center and scale each column of A to have mean 0 and variance 1 before fitting (as `sklearn.preprocessing.StandardScaler` does), you are implicitly applying Jacobi preconditioning to the normal equations. This is why data normalization matters so much for iterative solvers—and why scikit-learn does it by default.

The anytime property: In a real-time prediction pipeline, you might have a time budget of 1 ms. Plain CG won't converge in that time, but PCG reaches engineering accuracy ($< 1\%$ error) in 4 iterations. You can stop there and serve the prediction, then refine if time allows.

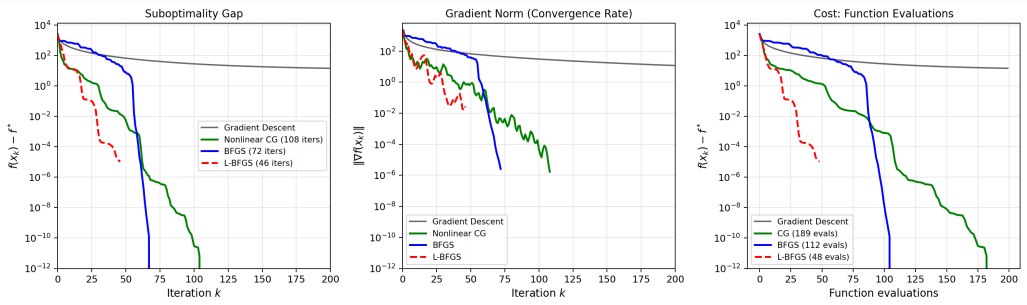
When to use Cholesky instead: For $n = 500$, Cholesky takes ~ 2 ms and gives the exact answer. CG is faster here only because the preconditioner is so effective. The real win for CG comes when $n \gg 10^3$ and A is sparse—then Cholesky fill-in makes it infeasible, while CG + preconditioner scales linearly.

Case Study 2: Logistic Regression (BFGS for Nonlinear Optimization)

Problem: Binary classification on $n = 5{,}000$ samples with $p = 50$ features (including correlated features). Minimize the regularized log-loss:

$$\min_{\mathbf{x}\in\mathbb{R}^{50}} \sum_{i=1}^{5000} \log(1 + e^{-y_i\, \mathbf{a}_i^T \mathbf{x}}) + \frac{\lambda}{2}\|\mathbf{x}\|^2$$

This is smooth, convex, but not quadratic—exactly where BFGS and L-BFGS shine.



Left: Suboptimality $f(\mathbf{x}_k) - f^*$. Gradient descent (black) barely moves; CG (green, 108 iters), BFGS (blue, 72 iters), and L-BFGS (red, 46 iters) converge orders of magnitude faster. **Center:** Gradient norm shows the superlinear convergence of BFGS/L-BFGS (the curve bends downward). **Right:** In total function evaluations, L-BFGS is the clear winner.

Case Study 2: What We Learn

Key observations:

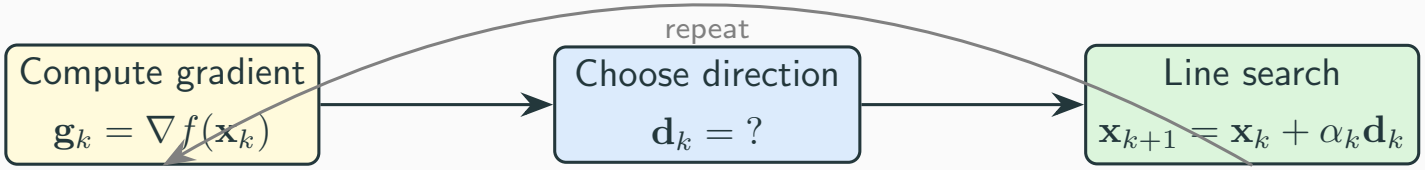
- Gradient descent is painfully slow. With step size $1/L$ (the theoretically optimal constant step size), GD hasn't converged after 500 iterations. The curvature of the log-loss varies across the domain, and a fixed step size can't adapt.
- BFGS and L-BFGS show superlinear convergence. Look at the gradient norm plot: the curves bend downward on the log scale near convergence. Each iteration makes more progress than the last. This is the Dennis–Moré condition in action—B_k is converging to the true Hessian along the search direction.
- L-BFGS beats full BFGS on this problem (46 vs. 72 iterations). The scaling heuristic $H_k^0 = \gamma_k I$ adapts the initial matrix at each step, which can outperform carrying forward a full H_k that may contain outdated curvature.
- This is exactly what scikit-learn does. `LogisticRegression(solver='lbfgs')` uses L-BFGS under the hood. For $n = 5{,}000$, $p = 50$, it converges in milliseconds.

What about larger problems? With $n = 10^6$ samples, computing the full gradient costs $O(np) = O(5 \times 10^7)$ per iteration. Stochastic gradient descent (SGD) approximates the gradient with a mini-batch of ~ 256 samples, reducing per-iteration cost by $\sim 4000\times$. But SGD converges much more slowly and only linearly—the tradeoff is explored in the next lecture.

Roadmap

What They Share

All three methods—CG, BFGS, L-BFGS—share a common structural pattern:



Common features:

1. Only gradient evaluations (no Hessian computation)
2. Wolfe line search ensures global convergence
3. Direction $\mathbf{d}_k$ is always a descent direction: $\mathbf{g}_k^T \mathbf{d}_k < 0$
4. Superlinear or better convergence (much faster than steepest descent)

The methods differ in how they choose $\mathbf{d}_k$—i.e., how much curvature information they use.

How They Differ: Direction Computation

	CG	BFGS	L-BFGS
Direction	$\mathbf{d}_k = -\mathbf{g}_k + \beta_k \mathbf{d}_{k-1}$	$\mathbf{d}_k = -H_k \mathbf{g}_k$	$\mathbf{d}_k = -H_k \mathbf{g}_k$ (implicit)
Curvature	Previous direction only	Full $n \times n$ matrix H_k	Last m pairs $(\mathbf{s}_i, \mathbf{y}_i)$
Memory	$O(n)$: 3–4 vectors	$O(n^2)$: dense matrix	$O(mn)$: $2m$ vectors
Cost/iter	$O(n)$	$O(n^2)$	$O(mn)$
Rate	Linear (nonquadratic); $\sqrt{\kappa}$ (quadratic)	Superlinear	Superlinear

Deep Dive: CG vs. BFGS

CG Advantages:

- $O(n)$ memory (crucial for $n > 10^6$)
- Finite convergence on quadratics (at most n steps)
- Krylov optimality: best use of mat-vec products
- Natural for linear systems $A\mathbf{x} = \mathbf{b}$
- Preconditionable

CG Limitations:

- Linear convergence on nonquadratic f

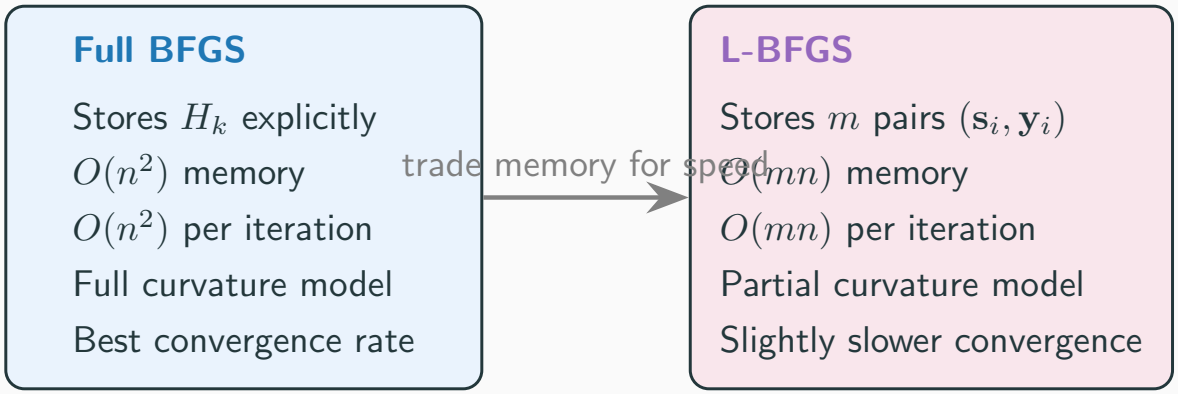
BFGS Advantages:

- Superlinear convergence (much faster)
- Hessian approximation "learns" over iterations
- Robust to inexact line search
- No restarts needed
- Guaranteed PD with Wolfe search

BFGS Limitations:

- $O(n^2)$ memory (prohibitive for large n)
- $O(n^2)$ per iteration

Deep Dive: BFGS vs. L-BFGS



When does L-BFGS match BFGS?

- Small problems ($n < 100$): nearly identical (use full BFGS anyway)
- Well-conditioned problems: $m = 5$–10 usually suffices
- Ill-conditioned problems: L-BFGS may need larger m or more iterations

Rule of thumb: L-BFGS with $m = 10$ is the default choice for large-scale unconstrained optimization. It is used in most machine learning optimizers when full-batch gradients are available.

The Connection: CG and L-BFGS with $m = 1$

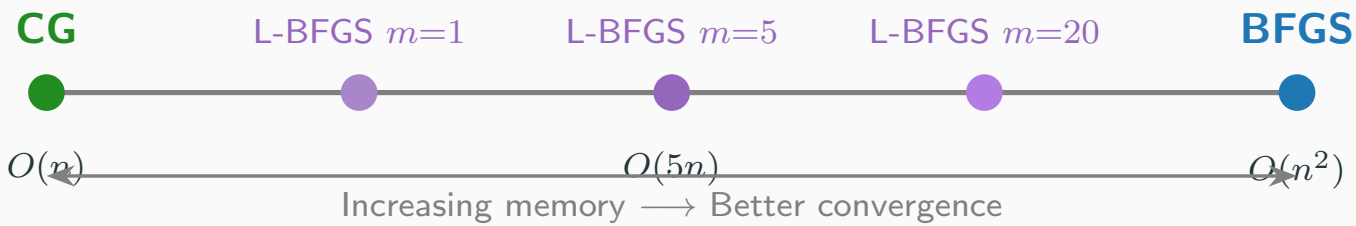
An interesting theoretical connection:

Theorem (Nocedal & Wright, §7.2)
L-BFGS with $m = 1$—i.e., the "memoryless BFGS" method that resets $H_k^0 = I$ and applies a single BFGS update—produces the search direction:

$$\mathbf{d}_{k+1} = -\nabla f_{k+1} + \frac{\nabla f_{k+1}^T \mathbf{y}_k}{\mathbf{y}_k^T \mathbf{d}_k} \mathbf{d}_k$$

which is exactly the Hestenes–Stiefel conjugate gradient formula. With exact line search, this reduces further to the Polak–Ribière formula.

This reveals a spectrum of methods:

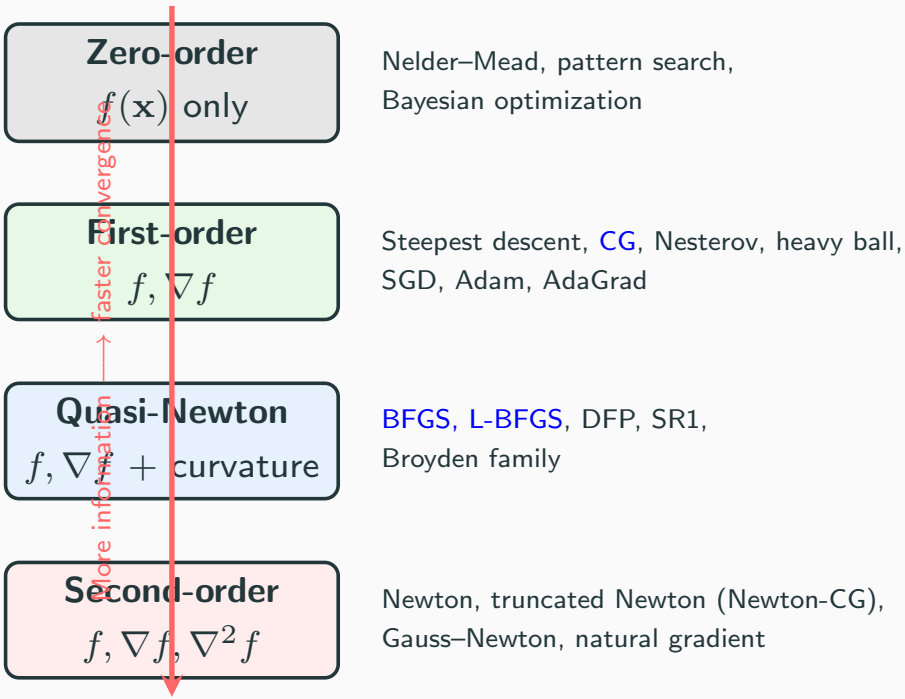


Roadmap

The Broader Landscape of Unconstrained Optimization

CG, BFGS, and L-BFGS are just one corner of a much larger world.

Methods are classified by what information they use:



More information per iteration $\Rightarrow$ faster convergence, but higher cost per step. The art is matching the method to the problem structure.

Where Does Nesterov Acceleration Fit?

Polyak heavy ball (1964): adds momentum to gradient descent:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k) + \beta(\mathbf{x}_k - \mathbf{x}_{k-1})$$

Nesterov accelerated gradient (1983): evaluates the gradient at a lookahead point:

$$\mathbf{y}_k = \mathbf{x}_k + \tfrac{k-1}{k+2}(\mathbf{x}_k - \mathbf{x}_{k-1}), \qquad \mathbf{x}_{k+1} = \mathbf{y}_k - \alpha \nabla f(\mathbf{y}_k)$$

Why does this matter? Nesterov achieves the optimal worst-case rate for smooth convex functions using only first-order information:

Method	**Rate**	**Optimal?**
Gradient descent	$O(1/k)$	No
Nesterov acceleration	$O(1/k^2)$	Yes (Nemirovsky & Yudin, 1983)
CG (on quadratics)	Finite ($\leq n$ steps)	Better (but quadratic only)
BFGS / L-BFGS	Superlinear	Much faster (but needs more per step)

The key distinction:

- Nesterov gives the best worst-case rate among pure gradient methods
- BFGS/L-BFGS get superlinear convergence in practice (faster, but not worst-case optimal)
- CG on quadratics: finite convergence (even better, but limited to quadratics)

What About Stochastic and Deep Learning Methods?

In machine learning, you often see SGD, Adam, AdaGrad instead of BFGS. Why don't we use BFGS for training neural networks?

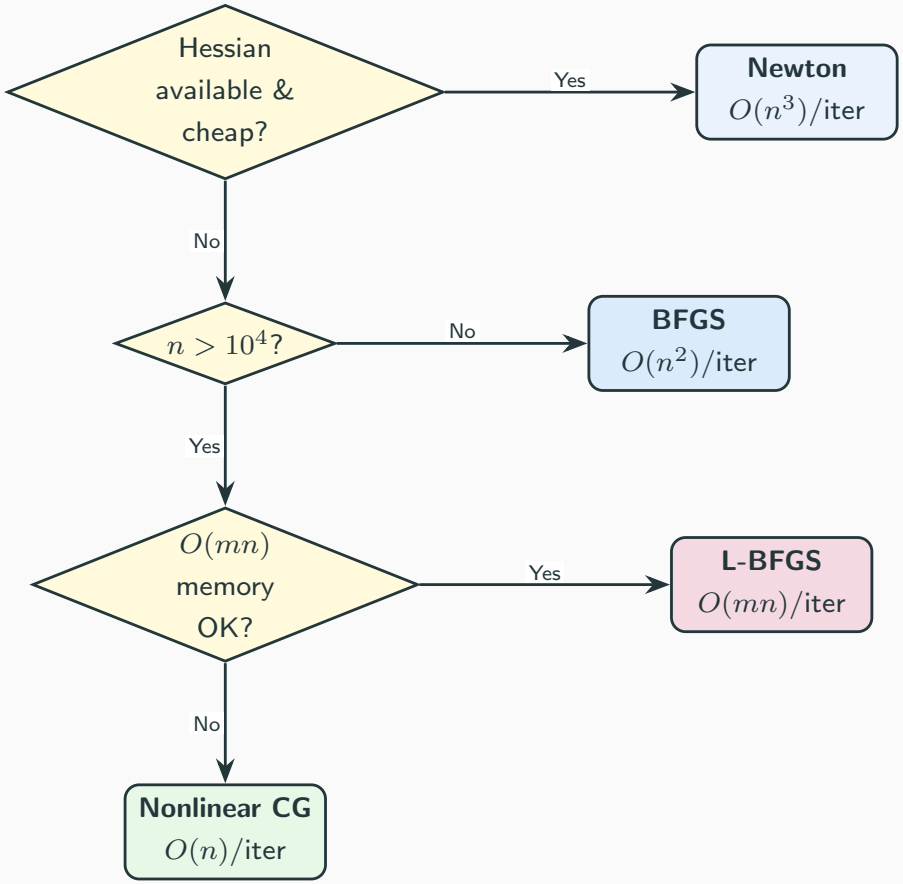
	Deterministic (this lecture)	**Stochastic** (deep learning)
Gradients	Exact $\nabla f(\mathbf{x})$	Noisy estimate $\hat{\nabla} f(\mathbf{x})$ from a mini-batch
Line search	Wolfe conditions ✓	Meaningless with noise ×
Curvature	Builds H_k from $(\mathbf{s}_k, \mathbf{y}_k)$ pairs	Noisy pairs ⇒ garbage H_k
Problem size	$n \lesssim 10^6$	$n \sim 10^8$–10^{11}
Loss landscape	Smooth, convex (or nearly)	Non-convex, many saddles

What ML uses instead:

- SGD + Nesterov momentum: the workhorse; optimal convergence rate under noise
- Adam: diagonal quasi-Newton in disguise—adapts a per-parameter "learning rate" using first and second moment estimates of the gradient
- L-BFGS: actually used when full-batch gradients are available (e.g., `scipy`, `scikit-learn`, some physics-informed neural nets)

Our methods are optimal for deterministic, moderate-scale problems. When gradients are noisy or $n > 10^8$, stochastic methods dominate—but the ideas (momentum ≈ conjugacy, Adam ≈ diagonal BFGS) trace directly back to what we've learned.

Decision Guide: Which Method Should I Use?



Default recommendation: If in doubt, start with L-BFGS ($m = 10$). It works well across a wide range of problem sizes and is the workhorse of modern optimization.

Summary: Three Methods at a Glance

Conjugate Gradient

- Uses A-conjugate directions to avoid SD's zigzag
- Finite convergence on quadratics; $\sqrt{\kappa}$ convergence factor
- $O(n)$ storage: the lightest method, for very large-scale problems
- PR+ variant recommended for nonlinear problems

BFGS

- Builds an approximate inverse Hessian from gradient differences
- Superlinear convergence; preserves positive definiteness with Wolfe search
- $O(n^2)$ storage and cost: for moderate-size problems
- Most popular quasi-Newton method

L-BFGS

- Implicitly approximates BFGS using m recent gradient pairs
- Superlinear convergence; two-loop recursion costs $O(mn)$
- $O(mn)$ storage: bridges the gap between CG and BFGS
- Default choice for large-scale optimization in practice

Roadmap

Exercise 1: CG by Hand

Exercise 1

Consider $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T Q\mathbf{x} + \mathbf{c}^T\mathbf{x}$ with $Q = \begin{pmatrix} 4 & 2 \\ 2 & 4 \end{pmatrix}$, $\mathbf{c} = \begin{pmatrix} -2 \\ -6 \end{pmatrix}$, starting from $\mathbf{x}_0 = (0, 0)^T$.

1. Verify $Q \succ 0$. Find the eigenvalues and the condition number κ.
2. Perform one iteration of CG: compute $\mathbf{g}_0$, $\mathbf{d}_0$, α_0, $\mathbf{x}_1$.
3. Compute $\mathbf{g}_1$, β_1, $\mathbf{d}_1$, α_1, $\mathbf{x}_2$. Verify $\mathbf{x}_2 = \mathbf{x}^*$.
4. Verify $\mathbf{d}_0^T Q\mathbf{d}_1 = 0$ (Q-conjugacy) and $\mathbf{g}_0^T\mathbf{g}_1 = 0$ (orthogonality).

Solution: Exercise 1

Solution

$\mathbf{g}_0 = \mathbf{c} = (-2, -6)^T, \quad \mathbf{d}_0 = (2, 6)^T.$

$Q\mathbf{d}_0 = (20, 28)^T, \quad \mathbf{d}_0^T Q\mathbf{d}_0 = 40 + 168 = 208.$

$\alpha_0 = \frac{4+36}{208} = \frac{40}{208} = \frac{5}{26}.$

$\mathbf{x}_1 = \frac{5}{26}(2, 6)^T = (\frac{5}{13}, \frac{15}{13})^T.$

$\mathbf{g}_1 = \mathbf{g}_0 + \alpha_0 Q\mathbf{d}_0 = (-2, -6)^T + \frac{5}{26}(20, 28)^T = (-2 + \frac{50}{13}, -6 + \frac{70}{13}) = (\frac{24}{13}, -\frac{8}{13})^T.$

Check: $\mathbf{g}_0^T \mathbf{g}_1 = -2 \cdot \frac{24}{13} + (-6) \cdot (-\frac{8}{13}) = \frac{-48+48}{13} = 0.$ ✓

$\beta_1 = \frac{(24/13)^2 + (8/13)^2}{40} = \frac{640/169}{40} = \frac{16}{169}.$

$\mathbf{d}_1 = -(\frac{24}{13}, -\frac{8}{13})^T + \frac{16}{169}(2, 6)^T = (-\frac{24}{13} + \frac{32}{169}, \ \frac{8}{13} + \frac{96}{169})^T.$

After completing the calculation: $\mathbf{x}_2 = (0, 1)^T = -Q^{-1}\mathbf{c} = \mathbf{x}^*.$ ✓

Exercise 2: BFGS Inverse Update

Exercise 2

Given $H_1 = I_2$, $\mathbf{s}_1 = (1, 0)^T$, $\mathbf{y}_1 = (2, 1)^T$.

1. Compute ρ_1, the matrices V_1 and W_1, and H_2 using the BFGS inverse update.
2. Verify $H_2\mathbf{y}_1 = \mathbf{s}_1$ (inverse secant condition).
3. Compute B_2 via the forward BFGS update and verify $B_2^{-1} = H_2$.
4. Is H_2 positive definite? What property of $\mathbf{s}_1, \mathbf{y}_1$ guarantees this?

Solution: Exercise 2

Solution

$\rho_1 = 1/(\mathbf{y}_1^T \mathbf{s}_1) = 1/2.$

$$V_1 = I - \tfrac{1}{2}(2,1)^T(1,0) = \begin{pmatrix} 0 & 0 \\ -1/2 & 1 \end{pmatrix},$$

$$W_1 = I - \tfrac{1}{2}(1,0)^T(2,1) = \begin{pmatrix} 0 & -1/2 \\ 0 & 1 \end{pmatrix}.$$

$$H_2 = W_1 V_1 + \tfrac{1}{2}\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 3/4 & -1/2 \\ -1/2 & 1 \end{pmatrix}.$$

$H_2 \mathbf{y}_1 = (3/2 - 1/2,\ -1 + 1)^T = (1,0)^T = \mathbf{s}_1$. ✓

$$B_2 = I - \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} 2 & 1 \\ 1 & 1/2 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 3/2 \end{pmatrix}.\ \det(B_2) = 2,$$

$B_2^{-1} = H_2$. ✓

Eigenvalues of H_2: $\frac{7 \pm \sqrt{17}}{8} \approx 1.39, 0.36 > 0$. PD guaranteed because $\mathbf{y}_1^T \mathbf{s}_1 = 2 > 0$.

Exercise 3: Method Comparison

Exercise 3

Consider the quadratic $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A\mathbf{x} - \mathbf{b}^T\mathbf{x}$ where A is $n \times n$ positive definite with condition number $\kappa = 10^4$.

1. How many iterations does steepest descent need (approximately) to reduce the A-norm error by a factor of 10^{-6}?
2. How many iterations does CG need (bound)?
3. If A has only 5 distinct eigenvalues (with large multiplicities), how many CG iterations does the theory guarantee?
4. For $n = 10^5$: which method would you choose among CG, BFGS, L-BFGS? Why?

Solution: Exercise 3

Solution

(1) SD: $\left(\frac{\kappa-1}{\kappa+1}\right)^k \leq 10^{-6}$, so $k \geq \frac{6\ln 10}{\ln(\kappa+1)/(\kappa-1)} \approx \frac{13.8}{2\times 10^{-4}} \approx 69{,}000.$

(2) CG: $2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k \leq 10^{-6}$, so $k \geq \frac{\ln(2\times 10^6)}{\ln(101/99)} \approx \frac{14.5}{0.02} \approx 725.$

(3) With only 5 distinct eigenvalues: CG converges in at most 5 iterations, regardless of n!

(4) For $n = 10^5$: BFGS requires $O(n^2) = O(10^{10})$ memory $\approx$ 80 GB—too large. L-BFGS with $m = 10$ requires $O(10 \times 10^5)$ memory $\approx$ 8 MB—fine. CG requires $O(n) \approx$ 800 KB. Choose L-BFGS (faster convergence than CG with affordable memory) or CG (if memory is extremely tight).

Looking Ahead

What We Covered Today

- **CG:** Motivation (PDEs, linear systems), A-conjugacy, Krylov subspaces, finite convergence, $\sqrt{\kappa}$ bound, eigenvalue clustering, preconditioning, nonlinear CG (FR, PR+), global convergence theory
- **BFGS:** Secant condition, rank-2 derivation, PD preservation, inverse update, DFP/Broyden family, global & superlinear convergence (Thms 6.5–6.6), self-correcting property
- **L-BFGS:** Two-loop recursion, memory parameter, CG$\leftrightarrow$L-BFGS connection

Reading:

- Nocedal & Wright: Ch. 5 (CG, Thms 5.1–5.8), Ch. 6 (QN, Thms 6.1–6.7), §7.2 (L-BFGS)
- Bazaraa, Sherali, Shetty: Sections 8.6–8.10
- Course notes: Chapters 14–15

Questions?

ISE 5406 — Optimization II

Virginia Tech, Spring 2026